DOCUVAULT – DIGITAL DOCUMENT REQUEST & RETRIEVAL SYSTEM

A PROJECT REPORT

Submitted by

NETHRA G

(240701358)

in partial fulfilment of the End Semester Examination for the course

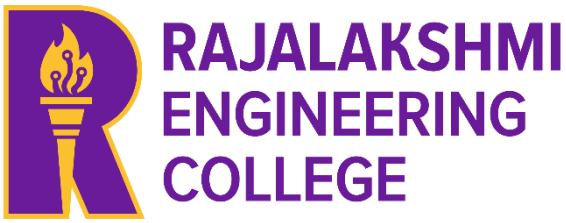
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "DocuVault – Digital Document Request & Retrieval System" is the bonafide work of Nethra G (240701358), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. J. Manoranjini
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Dr. J. Jinu Sophia
COURSE FACULTY

Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on 11 November 2026

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, Nethra G (240701358), hereby declare that the project report titled "DocuVault – Digital Document Request & Retrieval System", submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai
Date: 11 Novemeber 2026

SIGNATURE OF THE STUDENT
NETHRA G
240701358

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Mr. S.Meganadhan, and Chairperson Dr. M. Thangam Meganadhan for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S. N. Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. J. Manoranjini, Head of the Department, Department of Computer Science and Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Dr. J. Jinu Sophia , Associate Professor, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

NETHRA G

ABSTRACT

Educational institutions issue a large number of official documents every semester – bonafide certificates, conduct certificates, course-completion certificates, transcripts, duplicate mark statements and transfer certificates. In most colleges the process is still manual: students submit paper applications, the application travels between the department office, the Head of Department and the administrative office, and the student returns repeatedly to learn its status. There is no single record of who requested what, no tracking of turnaround time, no audit trail, and an employer or university that receives a paper certificate has no quick way to check that it is genuine.

DocuVault – Digital Document Request & Retrieval System – digitises the complete life cycle on Zoho Creator, a low-code platform that provides forms, reports, workflows, security, hosting and a mobile app out of the box, so the college's own staff can maintain it. The application contains six forms with a subform, ten reports (list, Kanban, calendar, chart, pivot and spreadsheet), three pages, ten workflows (form, approval, Blueprint, schedule and batch) and twelve Deluge scripts that validate requests, calculate fees, send notifications, generate a unique verification code and QR code for every issued document, log every access and expire documents automatically. Four roles – Administrator, Office Staff, Head of Department and Student (customer portal) – see only the data permitted by their profiles. The application integrates with Zoho Mail, Zoho Cliq, a public QR-code service through invokeurl and the Creator REST API.

The application was verified through unit, integration, role-based and user acceptance testing and was promoted through the Development, Stage and Production environments with audit trail, record-level access criteria and access logging enabled. Compared with the manual process, a typical request no longer needs repeated visits, the student sees live status, the office works from a single record with daily reminders for overdue items, and any issued document can be verified instantly by its code or QR code, which together reduce the manual steps per request by roughly two-thirds.

Keywords: *Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Role-Based Access Control; Document Management*

PROJECT AT A GLANCE

Attribute	Details
Project Title	DocuVault – Digital Document Request & Retrieval System
Application Domain	Education – Doct Vault – Digital Document Request
Student Name & Register No.	Nethra G – 240701358
Zoho Creator Edition	Trial Edition (as shown in the application header)
Application Name (Link Name)	DocuVault
Published / Live URL	nethra.g.2024.cse_rajalak322/digital-document-request-and-retrieval-system
Demo Credentials (per role)	Admin : nethra.g.2024.cse@rajalakshmi.edu.in
Number of Forms / Subforms	6 / 1 (Department, Student, Document_Type, Document_Request, Issued_Document, Access_Log / Request_Items)
Number of Reports (by type)	List: 5 | Kanban: 1 | Calendar: 1 | Pivot/Chart: 2 | Others: 1 (Spreadsheet)
Number of Pages / Dashboards	3 pages (Admin_Dashboard, Student_Home, Verify_Document)
Number of Workflows	Form: 5 | Schedule: 2 | Approval: 1 | Blueprint: 1 | Batch: 1
Deluge Functions / Custom Actions	3 functions / 2 custom actions (12 Deluge scripts in all)
Integrations	Zoho Mail, Zoho Cliq, QR-code REST API (invokeurl), Creator REST API v2
Roles & Profiles Defined	Administrator, Office Staff, HoD (Approver), Student (Customer Portal)
Environments Used	Development → Stage → Production
Date of Production Deployment	25 October 2026

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES xi
LIST OF ABBREVIATIONS xii
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 3
1.6 Organisation of the Report 4
LITERATURE SURVEY AND SYSTEM STUDY 5
2.1 Study of the Existing System 5
2.2 Limitations of the Existing System 6
2.3 Low-Code versus Traditional Development 7
2.4 Literature Survey 7
2.5 Emerging Trends in Low-Code Development 8
2.6 Proposed System 9
REQUIREMENT ANALYSIS 10
3.1 Stakeholders and User Roles 10
3.2 Functional Requirements 10
3.3 Non-Functional Requirements 12
3.4 Hardware and Software Requirements 12
3.5 Use Case Model 13
SYSTEM DESIGN 16
4.1 System Architecture 16
4.2 Data Model 17
4.3 Form Design 18
4.4 Process and Workflow Design 22
4.5 User Interface and Navigation Design 23
4.6 Access Control Design 24
IMPLEMENTATION 26
5.1 Application Setup 26
5.2 Forms and Data Capture 26

5.3 Reports 28
5.4 Pages and Dashboards 31
5.5 Deluge Scripting 32
5.6 Workflows and Automation 35
5.7 Integrations 36
5.8 User Management and Role-Based Access 38
5.9 AI and Analytics Features 38
5.10 Pro-Code Extension 39
5.11 Mobile and Portal Access 39
TESTING AND DEBUGGING 41
6.1 Testing Strategy 41
6.2 Test Cases 41
6.3 Debugging Deluge Scripts 42
6.4 Defect Log 43
6.5 User Acceptance Testing 43
DEPLOYMENT, SECURITY AND GOVERNANCE 45
7.1 Environments and Deployment Strategy 45
7.2 Performance Optimisation 46
7.3 Security Features 46
7.4 Governance and Administration 47
RESULTS AND DISCUSSION 48
8.1 Application Output 48
8.2 Achievement of Objectives 50
8.3 Comparison with the Existing System 50
8.4 Limitations of the Low-Code Approach 51
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 51
CONCLUSION AND FUTURE ENHANCEMENTS 52
9.1 Conclusion 52
9.2 Future Enhancements 52
9.3 Learning Outcomes and Career Relevance 52
REFERENCES 54
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 55
APPENDIX B – COURSE OUTCOME MAPPING 58
APPENDIX C – COMPLETE DELUGE SOURCE CODE 59
APPENDIX D – USER MANUAL 66
D.1 Accessing the Application 66
D.2 Instructions for the Student 66

D.3 Instructions for the Office Staff 66
D.4 Instructions for the HoD 67
D.5 Instructions for the Administrator 67
D.6 Instructions for a Verifier 67
APPENDIX E – EVALUATION RUBRIC 68
APPENDIX F – VIVA-VOCE RECORD SHEET 70

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 7
Table 2.2: Summary of Literature Survey 8
Table 3.1: Stakeholders and User Roles 10
Table 3.2: Functional Requirements 10
Table 3.3: Non-Functional Requirements 12
Table 3.4: Hardware and Software Requirements 12
Table 3.5: Use Case Description – Submit Document Request 13
Table 3.6: Use Case Description – Issue Document 14
Table 3.7: Use Case Description – Verify Document 14
Table 4.1: Entities and Relationships 17
Table 4.2: Field Specification – Department Form 18
Table 4.3: Field Specification – Student Form 18
Table 4.4: Field Specification – Document_Type Form 19
Table 4.5: Field Specification – Document_Request Form 19
Table 4.6: Field Specification – Request_Items Subform 20
Table 4.7: Field Specification – Issued_Document Form 21
Table 4.8: Field Specification – Access_Log Form 21
Table 4.9: Role–Permission Matrix 24
Table 5.1: Reports Implemented 29
Table 5.2: Deluge Scripts Implemented 32
Table 5.3: Workflows Implemented 35
Table 5.4: Integrations Implemented 37
Table 6.1: Test Cases 41
Table 6.2: Defect Log 43
Table 6.3: User Acceptance Feedback 44
Table 7.1: Deployment Checklist 45
Table 7.2: Performance Optimisation Results 46
Table 7.3: Security Controls Implemented 46
Table 8.1: Mapping of Objectives to Outcomes 50
Table 8.2: Existing System versus Proposed Application 50
Table A.1: Feature Coverage Checklist 55
Table B.1: Mapping of Course Outcomes to Report Chapters 58
Table E.1: Evaluation Rubric for the Individual Project 68
Table E.2: Performance Level Descriptors 69

Table F.1: Viva-Voce Questions and Responses 70
Table F.2: Marks Summary 70

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing System 6
Figure 3.1: Use Case Diagram of the Application 13
Figure 4.1: System Architecture of the Application..... 16
Figure 4.2: Entity–Relationship Diagram..... 17
Figure 4.3: Workflow / Process Flow for the Document Request 23
Figure 4.4: Navigation Structure and Wireframe of the Admin_Dashboard Page 24
Figure 5.1: Application Home and Navigation Sections..... 26
Figure 5.2: Document_Request Form in the Form Builder 27
Figure 5.3: Document_Request Form in Live Mode..... 28
Figure 5.4: Issued_Document Form in Live Mode 28
Figure 5.5: All_Requests List Report..... 29
Figure 5.6: Request_Pipeline Kanban Report..... 30
Figure 5.7: Due_Date_Calendar Report 30
Figure 5.8: Requests_by_Document_Type Chart Report..... 30
Figure 5.9: Monthly_Request_Trend Pivot Report..... 31
Figure 5.10: Admin_Dashboard Page 31
Figure 5.11: Student_Home Page on the Customer Portal 32
Figure 5.12: Verify_Document Page..... 32
Figure 5.13: Request Lifecycle Blueprint Configuration 36
Figure 5.14: Request Approval Process Configuration..... 36
Figure 5.15: Connection Setup and Postman Response 37
Figure 5.16: Roles and Permission Profiles Configuration 38
Figure 5.17: AI Text Recognition Field in Use..... 39
Figure 5.18: Application on Mobile and Customer Portal 40
Figure 6.1: Deluge Debug Output / Log..... 43
Figure 7.1: Environments and Publish to Production..... 45
Figure 7.2: Audit Trail and Administration Settings..... 47
Figure 8.1: Scenario Step 1 – Student Submits the Request..... 48
Figure 8.2: Scenario Step 2 – HoD Approves the Request..... 48
Figure 8.3: Scenario Step 3 – Staff Upload the Issued Documents 49
Figure 8.4: Scenario Step 4 – Student Downloads the Document 49
Figure 8.5: Scenario Step 5 – Public Verification of the Document..... 49
Figure 8.6: Scenario Step 6 – Dashboard Updated 49

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
DS	Deluge Script
ER	Entity–Relationship
FR	Functional Requirement
HoD	Head of the Department
HTML	HyperText Markup Language
ID	Identifier
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
NAD	National Academic Depository
NFR	Non-Functional Requirement
OAuth	Open Authorization
OCR	Optical Character Recognition
PDF	Portable Document Format
QR	Quick Response
RBAC	Role-Based Access Control
REST	Representational State Transfer
SLA	Service-Level Agreement
TC	Test Case
UC	Use Case
UI	User Interface
URL	Uniform Resource Locator
Zia	Zoho’s AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Official documents are the currency of academic life. A student who wishes to apply for higher studies, an internship, a passport, a bank loan or a scholarship must first obtain one or more certificates from the institution – a bonafide certificate, a conduct certificate, a course-completion certificate, a transcript, a duplicate mark statement or a transfer certificate. National initiatives such as DigiLocker [1] and the National Academic Depository [2] have raised the expectation that academic records can be issued, retrieved and verified digitally. At the level of an individual college office, however, the issuing process is still largely manual.

This project is built for the Administrative Office of a typical autonomous engineering college with about 4,000 to 5,000 students and eight academic departments. The application is modelled on the document service desk of such an institution; the department names, document types and fee amounts used as sample data are illustrative. Four groups of people are involved: the **students** who request documents, the **Heads of Department (HoDs)** who approve them, the **office staff** who prepare and issue them, and the **Administrator** (office superintendent) who supervises the service and the masters.

Today a student writes a paper application, obtains the signature of the HoD, carries it to the administrative office, and returns after a few days – often several times – to learn whether the certificate is ready. The clerk records the request in a register or spreadsheet, prepares the certificate in a word processor, takes a printout, and obtains the signature and seal of the authorised signatory. Requests peak at the end of each semester and during the admission and placement seasons, when the office is flooded with applications and the turnaround time grows. DocuVault replaces this paper trail with a single, role-based digital workflow on Zoho Creator.

1.2 Problem Statement

The college has no single system that records a document request from submission to retrieval. Requests are scattered across paper forms, registers and spreadsheets, approval depends on physical signatures, the student has no visibility of progress, and an external party that receives a certificate cannot easily confirm that it is genuine. The

result is delay, duplicated effort, lost applications and avoidable risk of forged documents. The specific pain points are:

- **No tracking:** students cannot see whether a request is received, approved, in preparation or ready, and must visit the office repeatedly.
- **Delayed, paper-based approvals:** an application waits in a physical tray until the HoD or Principal's office signs it.
- **Duplicate and untraceable requests:** the same certificate is requested more than once and there is no audit trail of who issued what and when.
- **No fast verification:** employers and universities verify a certificate by phone or e-mail, or simply trust the paper copy, which can be forged.
- **No management information:** volumes, pending requests and turnaround time are not available without manual counting.

1.3 Objectives

- To design a normalised data model and forms for students, departments, document types, document requests (with a subform of requested items), issued documents and access logs.
- To automate request validation, fee calculation, notification, verification-code generation and document expiry using Deluge scripting and workflows.
- To provide role-based dashboards and reports for the Administrator, office staff, HoDs and students, together with a customer portal for self-service retrieval.
- To integrate the application with Zoho Mail, Zoho Cliq and an external QR-code API for notification and verification, and to expose data through the Creator REST API.
- To implement a multi-level approval process and a Blueprint that enforces the request life cycle from submission to closure.
- To deploy the application securely to a production environment with governance controls such as roles, record-level access criteria, audit trail and access logging.

1.4 Scope of the Project

In scope. The application covers the complete life cycle of a document request: submission by a student through the customer portal, validation, fee calculation, approval by the HoD (and the Administrator for high-value documents), preparation

and issue of the document by office staff, secure retrieval by the student for a limited validity period, and public verification of the issued document using a code or QR code. It provides four user roles, six forms with one subform, ten reports, three pages, ten workflows and twelve Deluge scripts, and is accessible on the web, on the Zoho Creator mobile app and through the customer portal.

Out of scope. The project does not include an online payment gateway (fees are recorded as paid or unpaid by office staff), digital signatures with a certifying authority, integration with the college's examination or fee-management software, or generation of the certificate content itself; the office uploads the final signed PDF. These items are listed as future enhancements in Chapter 9.

1.5 Justification for a Low-Code Approach

Low-code application platforms let developers and trained business users build applications through visual modelling and a small amount of scripting instead of hand-written code for every layer [3]. Zoho Creator is such a platform: forms, reports, pages, workflows, approvals, role-based security, hosting and a mobile app are provided out of the box, and custom logic is written in the Deluge scripting language [4]. Industry analysts have projected that a large majority of new enterprise applications will use low-code or no-code technologies by 2025 [5]. For DocuVault, low-code is appropriate for the following reasons.

- **Development speed:** the data model, forms and reports were assembled in days rather than weeks, leaving time for testing and refinement.
- **Maintainability by non-programmers:** the college's own office staff can add a new document type or change a fee by editing a record or a form, without a software release.
- **Built-in hosting, security and mobile access:** roles, permission profiles, encrypted storage, audit trail and the Creator mobile app need no extra infrastructure.
- **Native approvals and life-cycle control:** the approval process and Blueprint features match the document-request workflow almost directly.
- **Cost:** a subscription (or the education programme) replaces the cost of servers, database administration and a full-stack development team.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In the existing system the document request is a paper-driven, multi-visit process. The student writes an application stating the document required and the purpose, and submits it at the department office. The HoD reviews and signs it. The student then carries the application to the administrative office, where a clerk enters the request in a register or spreadsheet and checks whether the details and fee receipt are complete; incomplete applications are returned to the student. For complete applications the clerk prepares the certificate in a word processor and takes a printout, which is signed and sealed by the Principal or HoD. The student collects the certificate in person, and any employer or university that later wishes to verify it must contact the office by phone or e-mail.

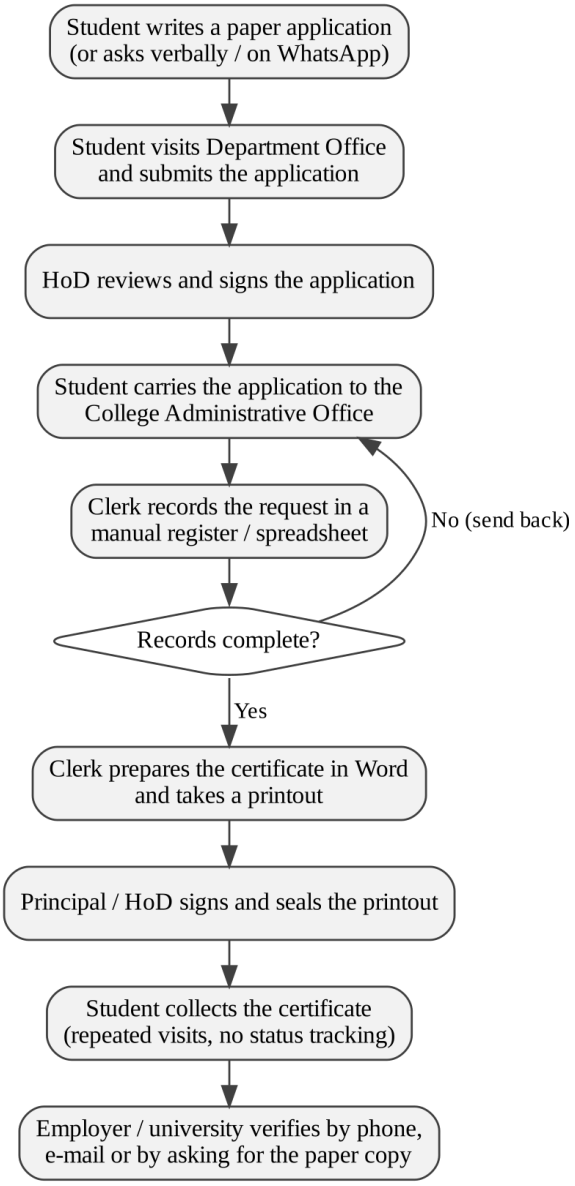


Figure 2.1: As-Is Process Flow of the Existing System

2.2 Limitations of the Existing System

- Students cannot track the status of a request and must make repeated physical visits.
- Approval and issue depend on physical signatures, so turnaround time is unpredictable and grows during peak periods.
- Requests are recorded in registers or spreadsheets, so duplicates, lost applications and data-entry errors are common.
- There is no audit trail of who prepared, approved or issued a document, or when it was handed over.
- Verification of a certificate is slow and manual, and paper copies can be forged.

- Management cannot see pending requests, overdue items or monthly volumes without manual counting.

2.3 Low-Code versus Traditional Development

Table 2.1 compares building DocuVault with a traditional pro-code stack (for example, a Java or Python web framework with a relational database) and with Zoho Creator. The effort figures are the author's planning estimates for an application of this size, not measured values.

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Approx. 8–10 weeks (design, back end, front end, testing)	Approx. 2–3 weeks (visual modelling plus Deluge scripts)
Skills required	Full-stack programming, database design, front-end development, DevOps	Data modelling, form and report design, basic Deluge scripting
Hosting & infrastructure	Server or cloud VM, database, SSL certificate and backups to be arranged	Hosted on Zoho cloud; SSL, backups and scaling are managed by the platform
Mobile application	Separate Android/iOS app or a responsive web design effort	Application is available in the Zoho Creator mobile app without extra coding
Security & access control	Authentication, role-based access and audit logging coded by the developer	Roles, permission profiles, record-level criteria and audit trail are configuration settings
Maintenance & change	Code change, build, test and redeploy for every new field or report	Drag-and-drop change in Development, tested in Stage and published to Production
Customisation limits	Practically unlimited	Bounded by platform capabilities, UI options and Deluge limits
Cost	Developer effort plus server and licence cost	Per-user subscription (or education programme); no server cost

2.4 Literature Survey

Five kinds of sources were reviewed: academic studies of low-code platforms, the Zoho Creator documentation, and existing digital document services. Table 2.2 summarises them.

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	R. Sanchis et al., 2020 [6]	Low-Code as Enabler of Digital Transformation in Manufacturing Industry	Low-code lets domain experts build and change applications quickly and reduces dependence on scarce IT developers.	Supports building the document service with the college's own office staff as citizen developers.
2	A. C. Bock and U. Frank, 2021 [7]	Low-Code Platform	Characterises low-code platforms (visual modelling, built-in runtime and deployment) and notes the risk of vendor lock-in.	Frames the choice of Zoho Creator and the limitations discussed in Section 8.4.
3	D. Di Ruscio et al., 2022 [8]	Low-Code Development and Model-Driven Engineering: Two Sides of the Same Coin?	Relates low-code to model-driven engineering: applications are produced from models of data, forms and processes.	Justifies the ER-model-first design of forms, lookups and subforms in Chapter 4.
4	Y. Luo et al., 2021 [9]	Characteristics and Challenges of Low-Code Development: The Practitioners' Perspective	Practitioners value rapid development but report limits in customisation, integration and learning resources.	Informs the workarounds recorded in Sections 5.10 and 8.4.
5	Zoho Corporation, Zoho Creator Help [4]	Zoho Creator and Deluge documentation	Describes forms, reports, pages, workflows, approvals, Blueprints, connections and environments.	Primary technical reference for the implementation in Chapter 5.
6	MeitY, Government of India [1]	DigiLocker	Issuer-authenticated digital documents that citizens store and share electronically.	Inspiration for digital issue and retrieval of institution-issued documents.
7	National Academic Depository [2]	National Academic Depository (NAD)	A digital repository of academic awards that supports online verification by third parties.	Inspiration for the public verification page and verification code.

2.5 Emerging Trends in Low-Code Development

Three trends influenced the design of DocuVault. First, **citizen development**: platforms such as Zoho Creator allow trained business users to maintain applications, so DocuVault keeps document types, fees, validity periods and approval levels in master forms that office staff can edit without touching a script. Second, **AI-assisted building**: Zia, the AI assistant of Zoho, can generate form structures and Deluge snippets from

natural-language prompts, and AI fields can extract text from images; DocuVault uses both, and every generated script was reviewed and tested before use. Third, **hyper-automation** – chaining validation, approvals, notifications, scheduled jobs and integrations – is what turns a set of forms into a self-running service, and it is the pattern followed by the workflows of Chapter 5.

2.6 Proposed System

DocuVault is a Zoho Creator application that digitises the whole request-to-retrieval cycle. A student submits a request on the customer portal; the system validates it, calculates the fee, acknowledges it by e-mail and alerts the office. The HoD (and, for high-value documents, the Administrator) approves it through an approval process, and the office moves it through a Blueprint-controlled life cycle until the signed PDF is uploaded. At that moment the system generates a unique verification code and QR code, sets a validity date and notifies the student, who downloads the document from the portal while it is valid. Every download, verification and expiry is logged, and an external verifier can confirm any document on a public page. Each limitation in Section 2.2 is addressed as follows: tracking by the portal and status reports; approval delays by the approval process and SLA reminders; duplicates by validation scripts; the missing audit trail by the access log and audit trail; slow verification by the verification page; and lack of management information by the dashboard and reports.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator	Office superintendent who owns the service	Manage users and master data, approve high-value documents, view all data, dashboards and access logs
Office Staff	Document clerks of the administrative office	Review requests, record payment, prepare and upload documents, move requests through the Blueprint
HoD (Approver)	Head of the student's department	Approve or reject requests of own department through the approval process
Student (Portal User)	Current student or alumnus registered on the customer portal	Submit requests, track status, download own valid documents

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall allow a student to submit a document request selecting one or more document types, the number of copies, the purpose and the required-by date.	Student	High
FR-02	The system shall reject a request that duplicates an open request for the same document type or whose required-by date is earlier than the processing time.	System	High
FR-03	The system shall auto-fill the department, e-mail and HoD details from the selected student and calculate the fee automatically.	System	High
FR-04	The system shall send an acknowledgement e-mail to the student and an alert to the office when a request is submitted.	System	Medium
FR-05	The system shall route requests for approval to the HoD, and additionally to the Administrator for high-value documents, and record the decision.	HoD, Administrator	High
FR-06	The system shall control the request life cycle (Submitted to Closed) so that only permitted transitions are made by permitted roles.	Office Staff	High

FR ID	Requirement	Role(s)	Priority
FR-07	The system shall allow office staff to upload the final PDF against an approved request and record the issue date.	Office Staff	High
FR-08	The system shall generate a unique verification code, a validity date and a QR code for every issued document.	System	High
FR-09	The system shall let a student view the status of own requests and download own issued documents only while they are valid.	Student	High
FR-10	The system shall log every download, verification and expiry event of an issued document.	System	Medium
FR-11	The system shall automatically expire issued documents after their validity period.	System	Medium
FR-12	The system shall provide a public page where a document can be verified using its verification code.	Verifier	High
FR-13	The system shall remind the office every day about requests that have exceeded their processing time.	Office Staff, Administrator	Medium
FR-14	The system shall display a dashboard of request volume, pending approvals, issued documents and turnaround time.	Administrator	Medium
FR-15	The system shall archive requests that have been closed for more than twelve months.	System	Low
FR-16	The system shall expose issued-document data to authorised external programs through the Creator REST API.	Administrator	Low

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Reports with up to 5,000 request records shall open within 3 seconds when filtered by default criteria.
NFR-02	Security	Users shall access only the data permitted by their role; a student shall see only own requests and documents.
NFR-03	Usability	The application shall be usable on the web and on the Zoho Creator mobile app, and a request shall be submittable in under two minutes.
NFR-04	Availability	The application shall be available round the clock through Zoho's cloud hosting without any server maintenance by the college.
NFR-05	Maintainability	Masters (document types, fees, validity) shall be editable by the Administrator without changing scripts; changes shall be made in Development and promoted through Stage.
NFR-06	Auditability	All record changes shall be traceable through the audit trail, and every document access through the Access_Log form.
NFR-07	Authenticity	Every issued document shall carry a verification code that cannot be guessed from the request or document number.
NFR-08	Privacy	The public verification page shall reveal only the student name, document type, issue date and status.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Platform	Zoho Creator Trial Edition
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox (latest version)
API testing tool	Postman
Other Zoho services	Zoho Mail (notifications), Zoho Cliq (office alerts)
External service	QR-code generation API (public HTTP service)
Mobile	Zoho Creator mobile app (Android / iOS) and the customer portal
Client hardware	Intel Core i3/i5 or above, 8 GB RAM, internet connection

3.5 Use Case Model

Figure 3.1 shows the use cases of the five actors. The Student, Office Staff, HoD and Administrator are authenticated users; the Verifier is an external person (for example, an employer or university) who uses only the public verification page. Three key use cases are described in Tables 3.5 to 3.7.

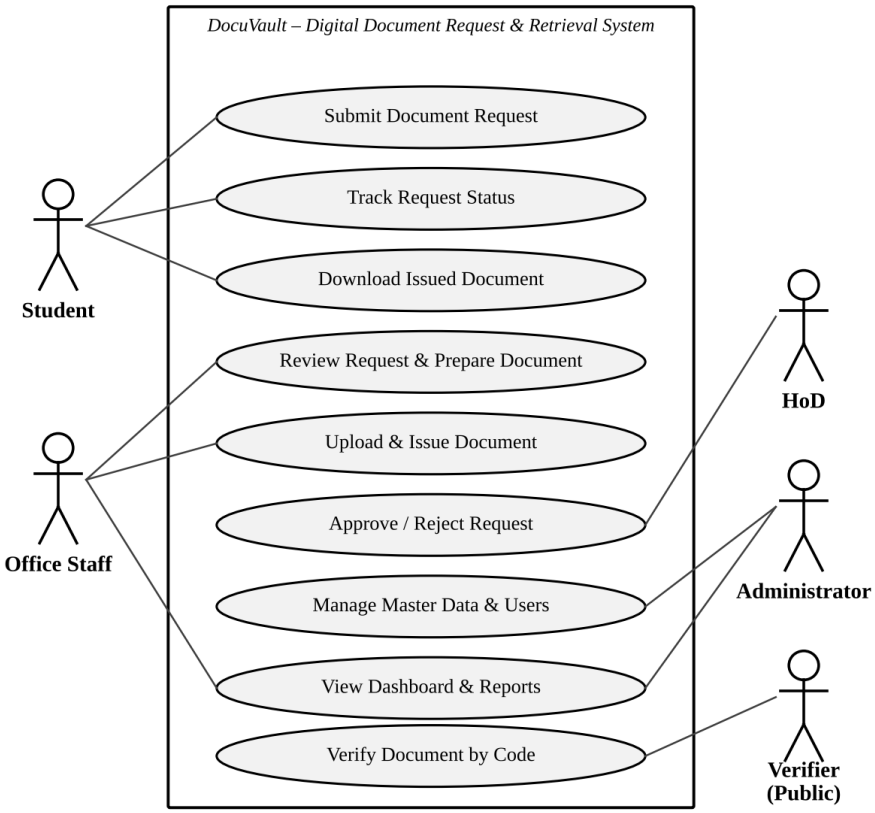


Figure 3.1: Use Case Diagram of the Application

Table 3.5: Use Case Description – Submit Document Request

Use Case ID	UC-01
Use Case Name	Submit Document Request
Actor(s)	Student
Pre-condition	The student is registered in the Student form and has an active customer-portal login.

Main Flow	1. The student opens the portal and selects *Request a Document*. 2. The student selects own name; department, e-mail and HoD are auto-filled. 3. The student adds one or more rows (document type and copies) to the subform; the fee is calculated. 4. The student enters the purpose, required-by date and delivery mode, attaches the ID card image and ticks the declaration. 5. The student submits the form; validation script DS-01 runs. 6. On success an acknowledgement e-mail is sent and the office is alerted.
Alternate Flow	At step 5, if a duplicate open request exists or the required-by date is too early, an alert is shown and the submission is cancelled.
Post-condition	A Document_Request record with status Submitted exists and is visible in My_Requests and the office reports.
Related FR	FR-01, FR-02, FR-03, FR-04

Table 3.6: Use Case Description – Issue Document

Use Case ID	UC-02
Use Case Name	Issue Document
Actor(s)	Office Staff, System
Pre-condition	The request has Approval_Status = Approved and the fee is marked Paid or Waived.
Main Flow	1. Staff moves the request to *In Preparation* and prepares the signed PDF. 2. Staff adds an Issued_Document record against the request and uploads the PDF. 3. Script DS-05 generates the verification code, validity date and QR code and e-mails the student. 4. When every requested item has an issued document, staff performs the *Issue Document* transition; DS-06 checks completeness.
Alternate Flow	If an item has no issued document at step 4, the transition is blocked with an alert.
Post-condition	The request status is Issued and the student can download the document from My_Documents.
Related FR	FR-06, FR-07, FR-08, FR-09

Table 3.7: Use Case Description – Verify Document

Use Case ID	UC-03
Use Case Name	Verify Document
Actor(s)	Verifier (public user)

Pre-condition	The verifier has the verification code printed on the document or its QR code.
Main Flow	1. The verifier opens the public Verify_Document page (directly or by scanning the QR code). 2. The verifier enters the code and presses *Verify*. 3. Function DS-10 looks up the code, returns the student name, document type, issue date and status, and writes a Verified entry in Access_Log.
Alternate Flow	If no document matches the code, the page shows “No document found”; if the document is expired or revoked, the status is displayed.
Post-condition	The verifier knows whether the document is genuine and active; the check is logged.
Related FR	FR-10, FR-12

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

DocuVault follows the layered architecture shown in Figure 4.1. The **user and channel layer** consists of the students (customer portal and mobile app), office staff, HoDs and the Administrator (web), and external verifiers who use a public page. The **presentation layer** is built with the UI Builder and the report engine: three pages and ten reports. The **application-logic layer** holds the forms and their validations, Deluge scripts and functions, form, schedule and batch workflows, the approval process and the Blueprint; role-based access control cuts across this layer through roles, permission profiles and record-level criteria. The **data layer** stores the six forms and the subform in Zoho Creator's managed database, including uploaded files. The **integration layer** connects the application to Zoho Mail and Zoho Cliq, to a public QR-code API through *invokeurl*, and to external programs through the Creator REST API.

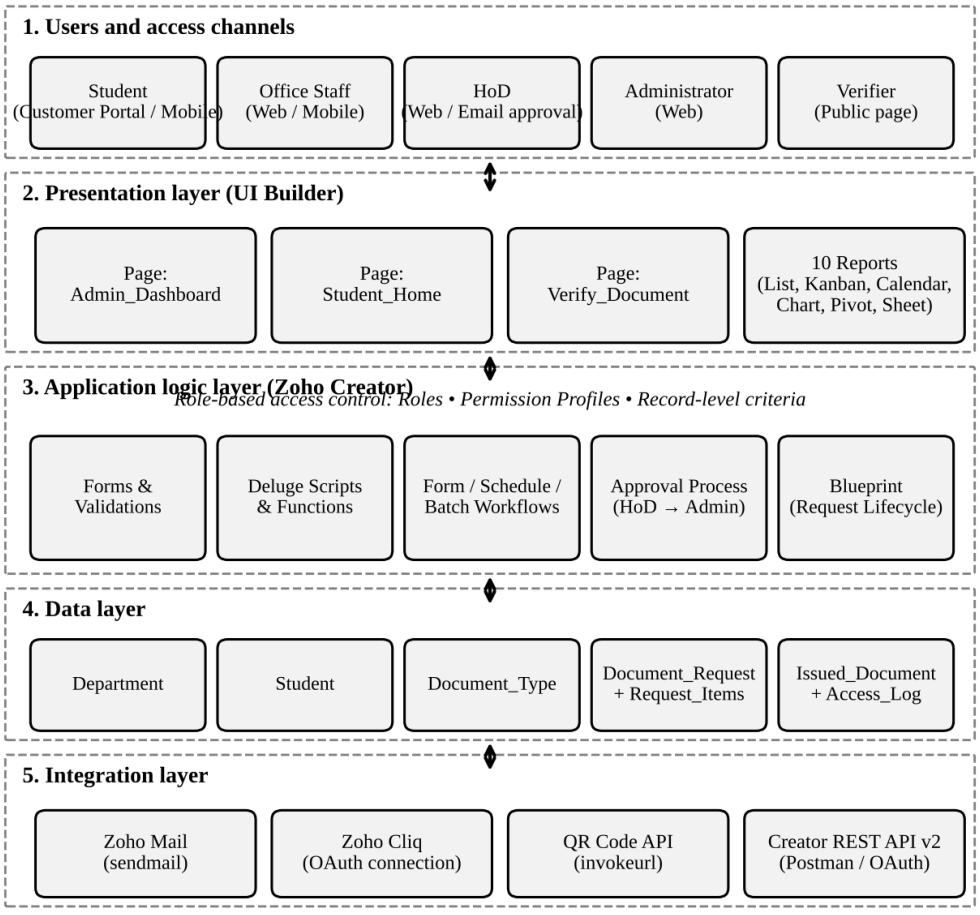


Figure 4.1: System Architecture of the Application

4.2 Data Model

The data model has seven entities (Figure 4.2, Table 4.1). A **Department** has many **Students**; a Student raises many **Document_Requests**. A request contains one or more rows of **Request_Items**, each pointing to a **Document_Type**; this parent–child relationship is implemented as a subform because the rows have no meaning outside the request. For every approved request the office creates one or more **Issued_Document** records, each linked to the request and to its Document_Type, and every download, verification or expiry of an issued document is recorded in **Access_Log**. One-to-many relationships between independent forms are implemented through lookup fields so that the child form stores the parent's record ID, which keeps the data normalised and allows criteria such as Student.Email to be used for security.

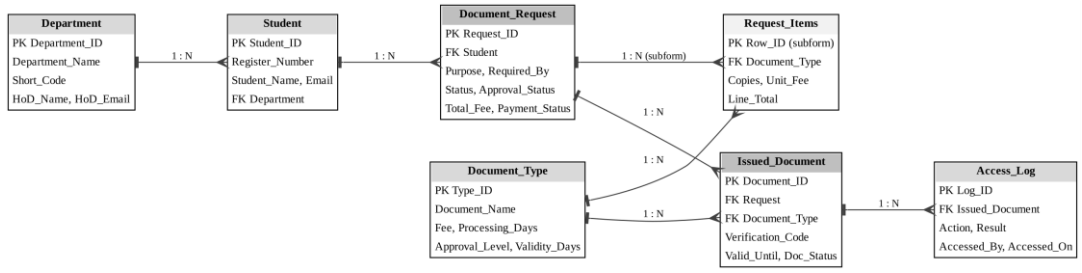


Figure 4.2: Entity–Relationship Diagram

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Department	Department_ID, Department_Name, HoD_Email	Student	One-to-Many	Lookup in Student
Student	Student_ID, Register_Number, Email	Document_Request	One-to-Many	Lookup in Document_Request
Document_Type	Type_ID, Document_Name, Fee, Approval_Level	Request_Items, Issued_Document	One-to-Many	Lookup in Request_Items and Issued_Document
Document_Request	Request_ID, Status, Approval_Status, Total_Fee	Request_Items	Parent–Child	Subform
Request_Items (subform)	Document_Type, Copies, Unit_Fee, Line_Total	Document_Request	Parent–Child	Subform rows
Issued_Document	Document_ID, Verification_Code, Valid_Until	Document_Request, Access_Log	One-to-Many	Lookup in Issued_Document and Access_Log
Access_Log	Log_ID, Action, Accessed_On	Issued_Document	Many-to-One	Lookup

4.3 Form Design

One field-specification table is given for each form (Tables 4.2 to 4.8). Field types are named exactly as on the Zoho Creator platform. Fields marked as auto-filled or read-only are populated by workflow scripts and cannot be edited by students.

Table 4.2: Field Specification – Department Form

Field Label	Link Name	Field Type	Required	Validation / Default	Remarks
Department ID	Department_ID	Auto Number	—	Prefix DEPT-	System generated
Department Name	Department_Name	Single Line	Yes	No duplicate values	e.g., Computer Science and Engineering
Short Code	Short_Code	Single Line	Yes	Maximum 6 characters	e.g., CSE
HoD Name	HoD_Name	Name	Yes	—	Shown in approval mails
HoD Email	HoD_Email	Email	Yes	Valid e-mail format	Approver of the approval process
Active	Active	Decision Box	—	Default: checked	Unchecked departments are hidden

Table 4.3: Field Specification – Student Form

Field Label	Link Name	Field Type	Required	Validation / Default	Remarks
Student ID	Student_ID	Auto Number	—	Prefix STU-	System generated
Register Number	Register_Number	Single Line	Yes	Exactly 9 digits; no duplicates	e.g., 240701358
Student Name	Student_Name	Name	Yes	—	First and last name
Department	Department	Lookup	Yes	Lookup to Department	One-to-many
Year of Study	Year_of_Study	Drop Down	Yes	I, II, III, IV	—
Batch	Batch	Single Line	Yes	Format YYYY–YYYY	e.g., 2024–2028
Email	Email	Email	Yes	No duplicate values	Portal login and notifications
Phone	Phone	Phone	No	10 digits	—
Photo	Photo	Image	No	JPG / PNG	—

Field Label	Link Name	Field Type	Required	Validation / Default	Remarks
Status	Status	Drop Down	Yes	Default: Active	Active / Alumni / Discontinued

Table 4.4: Field Specification – Document_Type Form

Field Label	Link Name	Field Type	Required	Validation / Default	Remarks
Type ID	Type_ID	Auto Number	—	Prefix DT-	System generated
Document Name	Document_Name	Single Line	Yes	No duplicate values	e.g., Bonafide Certificate
Description	Description	Multi Line	No	—	Shown on the request form
Fee (Rs.)	Fee	Currency	Yes	Value ≥ 0	Illustrative sample values
Processing Days	Processing_Days	Number	Yes	1 to 15	Service-level target
Approval Level	Approval_Level	Radio	Yes	None / HoD / HoD + Administrator; default HoD	Drives the approval process
Validity (Days)	Validity_Days	Number	Yes	Default: 30	Download validity of an issued copy
Active	Active	Decision Box	—	Default: checked	—

Table 4.5: Field Specification – Document_Request Form

Field Label	Link Name	Field Type	Required	Validation / Default	Remarks
Request ID	Request_ID	Auto Number	—	Prefix DVR-	System generated
Student	Student	Lookup	Yes	Lookup to Student	Record-level security key
Department	Department	Lookup	—	Auto-filled (DS-02)	Read-only
Student Email	Student_Email	Email	—	Auto-filled (DS-02)	Read-only
HoD Email	HoD_Email	Email	—	Auto-filled (DS-02)	Hidden; used by approval
Request Date	Request_Date	Date-Time	—	Default: zoho.currenttime	Read-only

Field Label	Link Name	Field Type	Required	Validation / Default	Remarks
Purpose	Purpose	Drop Down	Yes	Higher Studies, Employment, Passport, Scholarship, Bank Loan, Internship, Other	—
Purpose Details	Purpose_Details	Multi Line	Conditional	Mandatory when Purpose = Other	Layout rule shows/hides
Required By	Required_By	Date	Yes	Not before today + processing days (DS-01)	—
Delivery Mode	Delivery_Mode	Radio	Yes	Digital Download / Collect from Office; default Digital Download	—
ID Card Image	ID_Card_Image	Image	No	JPG / PNG	Input to the AI field
ID Card Text	ID_Card_Text	AI – Text Recognition	—	Extracted from ID_Card_Image	Staff compares with register number
Request Items	Request_Items	Subform	Yes	At least one row	See Table 4.6
Total Fee	Total_Fee	Formula	—	Sum of Line_Total	Currency, read-only
Payment Status	Payment_Status	Drop Down	Yes	Default: Unpaid; Unpaid / Paid / Waived	Updated by staff
Approval Status	Approval_Status	Drop Down	—	Default: Pending; Pending / Approved / Rejected	Set by approval process
Status	Status	Drop Down	Yes	Default: Submitted	Controlled by the Blueprint
Remarks	Remarks	Multi Line	No	—	Office remarks, rejection reason
Declaration	Declaration	Decision Box	Yes	Must be checked	Confirms details are correct
Archived	Archived	Decision Box	—	Default: unchecked	Set by batch workflow

Table 4.6: Field Specification – Request_Items Subform

Field Label	Link Name	Field Type	Required	Validation / Default	Remarks
Document Type	Document_Type	Lookup	Yes	Lookup to Document_Type	Active types only
Copies	Copies	Number	Yes	1 to 5; default 1	—

Field Label	Link Name	Field Type	Required	Validation / Default	Remarks
Unit Fee	Unit_Fee	Currency	—	Set from Document_Type.Fee	Read-only
Line Total	Line_Total	Formula	—	Copies × Unit_Fee	Read-only

Table 4.7: Field Specification – Issued_Document Form

Field Label	Link Name	Field Type	Required	Validation / Default	Remarks
Document ID	Document_ID	Auto Number	—	Prefix DOC-	System generated
Request	Request	Lookup	Yes	Approved requests only	Parent request
Student	Student	Lookup	—	Auto-filled from Request	Read-only; security key
Document Type	Document_Type	Lookup	Yes	Lookup to Document_Type	—
Document File	Document_File	File Upload	Yes	PDF only	Final signed document
Issue Date	Issue_Date	Date	Yes	Default: today	—
Valid Until	Valid_Until	Date	—	Issue date + Validity_Days (DS-05)	Read-only
Verification Code	Verification_Code	Single Line	—	Unique; generated by DS-05	Read-only
Verification QR	Verification_QR	Image	—	Generated by DS-05	Printed on the document
Issued By	Issued_By	Single Line	—	Default: zoho.loginuser	Read-only
Download Count	Download_Count	Number	—	Default: 0	Incremented by DS-11
Status	Doc_Status	Drop Down	Yes	Default: Active; Active / Expired / Revoked	Changed by DS-08

Table 4.8: Field Specification – Access_Log Form

Field Label	Link Name	Field Type	Required	Validation / Default	Remarks
Log ID	Log_ID	Auto Number	—	Prefix LOG-	System generated
Issued Document	Issued_Document	Lookup	Yes	Lookup to Issued_Document	—

Field Label	Link Name	Field Type	Required	Validation / Default	Remarks
Action	Action	Drop Down	Yes	Downloaded / Verified / Expired / Revoked	—
Accessed By	Accessed_By	Single Line	Yes	User ID or “Public Verifier”	—
Accessed On	Accessed_On	Date-Time	—	Default: zoho.currenttime	Read-only
Result	Result	Single Line	No	e.g., Success, Valid, Expired	—

4.4 Process and Workflow Design

Figure 4.3 shows the process flow of a document request. Shading identifies who acts: students (white), system scripts (light grey), office staff (medium grey) and the HoD or Administrator (dark grey). Deluge scripts DS-01, DS-04, DS-05, DS-06 and DS-11 run at the points marked, and the two schedule workflows DS-07 (SLA reminder) and DS-08 (expiry) run independently of the flow every day. The request life cycle is held in a Blueprint with the stages Submitted, Under Review, Pending Approval, Approved, In Preparation, Issued, Closed and Rejected; each transition is available only to the permitted role.

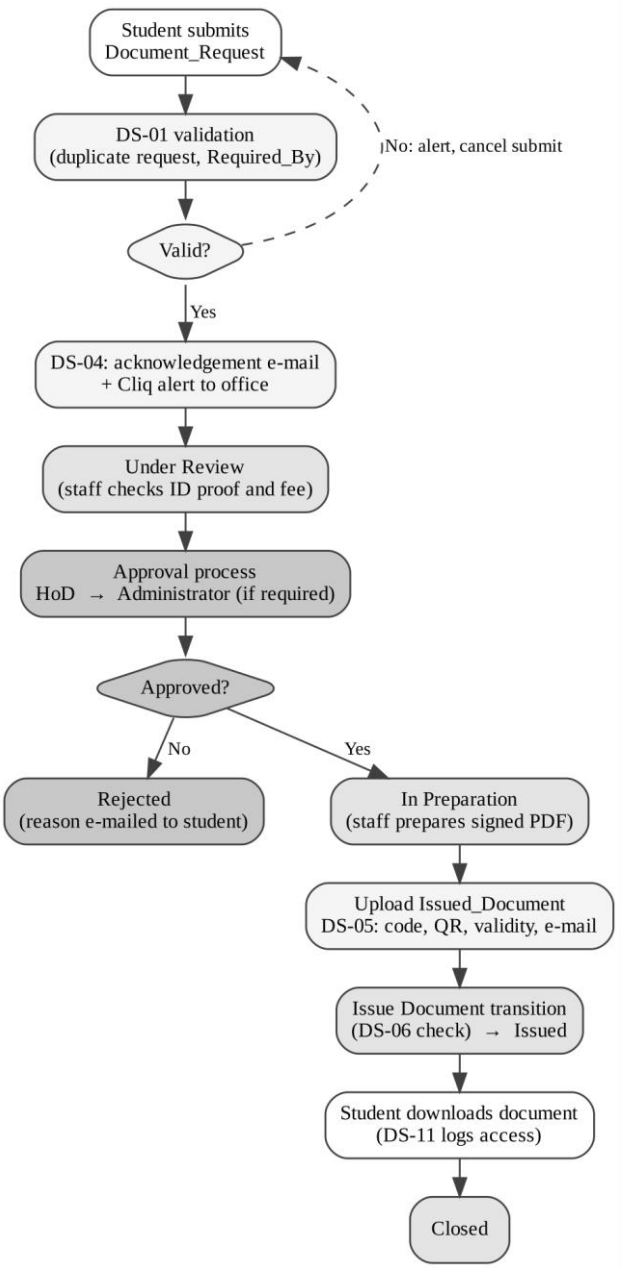


Figure 4.3: Workflow / Process Flow for the Document Request

4.5 User Interface and Navigation Design

The navigation is organised into six sections – Dashboard, Requests, Documents, Analytics, Masters and Student Portal – and each role sees only the sections permitted by its profile. Figure 4.4 shows the navigation structure and the wireframe of the Admin_Dashboard page, which combines four KPI panels, three charts and an embedded report of overdue requests.

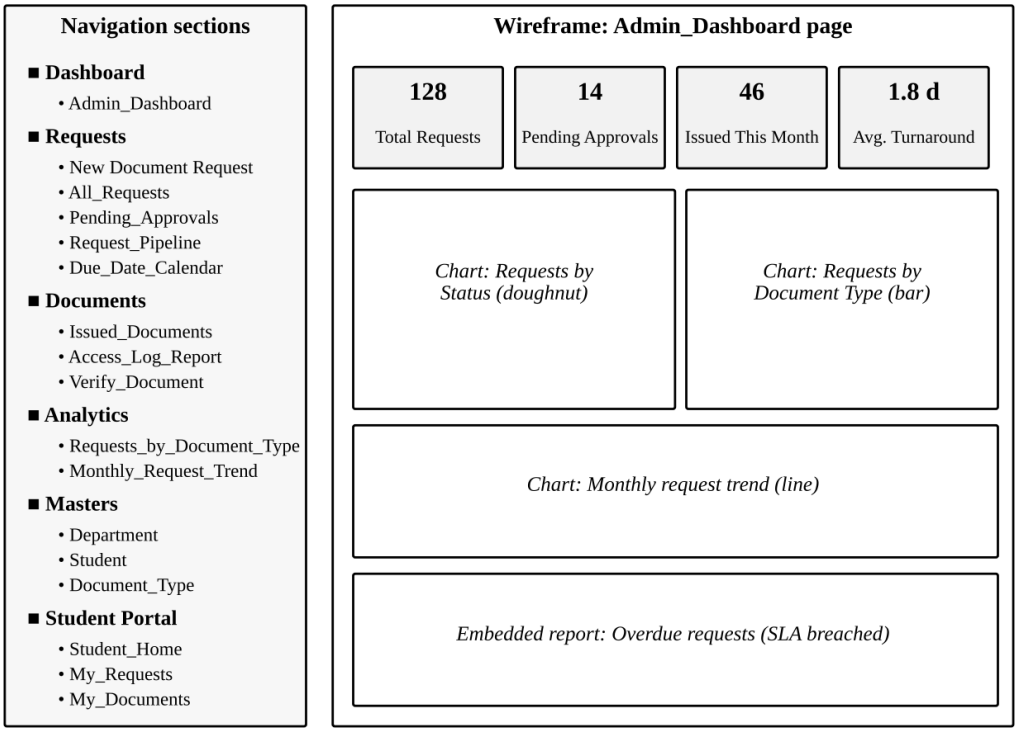


Figure 4.4: Navigation Structure and Wireframe of the Admin_Dashboard Page

4.6 Access Control Design

Table 4.9 defines the permission matrix that Chapter 5 implements. For students, "own" means record-level criteria that match the logged-in portal user's e-mail with the Student record; for HoDs, "own dept" means requests whose department's HoD e-mail matches the logged-in user.

Table 4.9: Role–Permission Matrix

Component	Administrator	Office Staff	HoD	Student (Portal)
Form: Department	A E V D	V	V	—
Form: Student	A E V D	A E V	V (own dept)	—
Form: Document_Type	A E V D	V	V	V
Form: Document_Request	A E V D	E V	V (own dept)	A V (own)
Form: Issued_Document	A E V D	A E V	V (own dept)	V (own)
Form: Access_Log	V	V	—	—
Report: All_Requests	V	V	—	—
Report: Pending_Approvals	V	—	V (own dept)	—
Report: My_Requests / My_Documents	—	—	—	V (own)
Page: Admin_Dashboard	V	V	—	—

Component	Administrator	Office Staff	HoD	Student (Portal)
Page: Student_Home	—	—	—	V
Page: Verify_Document	V	V	V	V (public)

A = Add, E = Edit, V = View, D = Delete, — = No access.

CHAPTER 5
IMPLEMENTATION

This chapter describes each component of DocuVault as built on Zoho Creator. For every component the purpose, the configuration choices and the evidence (screenshot or code) are given; each section maps to a syllabus unit and to Appendix A.

5.1 Application Setup

The application was created in the Zoho Creator workspace of the author under the link name **DocuVault**. The initial structure – forms, a first set of fields and sample reports – was generated from a natural-language prompt with the Zia AI application builder, and was then reviewed and reworked manually: field types, lookups, validations and every script were checked against the design of Chapter 4. The application menu is organised into six sections (Dashboard, Requests, Documents, Analytics, Masters and Student Portal) that follow the navigation design of Figure 4.4. A common theme (colour, logo and application name) was applied so that the staff view and the customer portal look consistent, and the application link name and form link names follow a single naming convention (capitalised words joined by underscores).

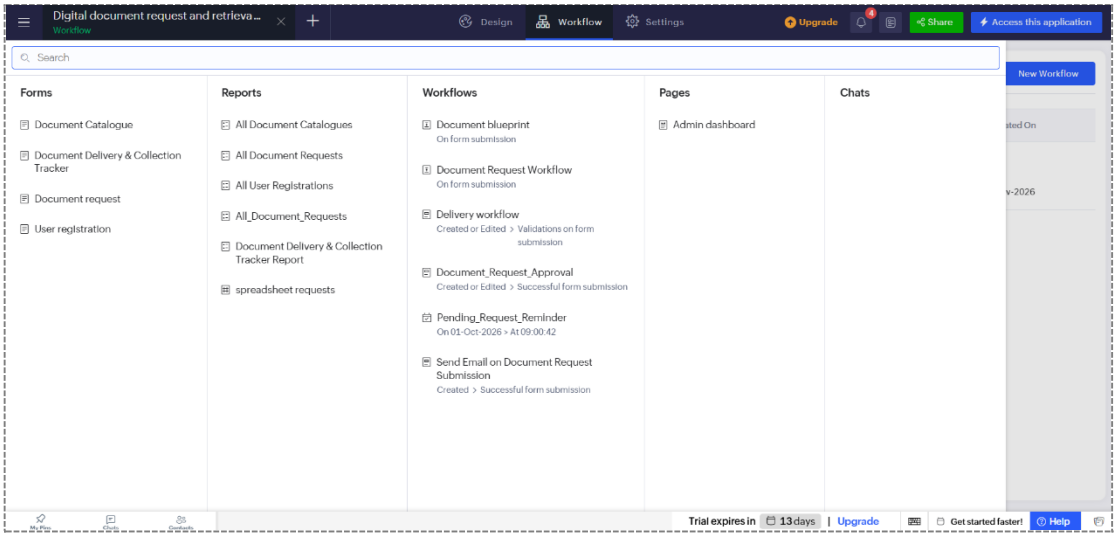


Figure 5.1: Application Home and Navigation Sections

5.2 Forms and Data Capture

Six forms and one subform were built (Tables 4.2 to 4.8). Nineteen distinct field types are used, including Single Line, Auto Number, Name, Email, Phone, Drop Down, Radio, Decision Box, Date, Date-Time, Number, Currency, Formula, Lookup,

Subform, File Upload, Image, Multi Line and the AI Text Recognition field. Key design choices are as follows.

- **Lookups:** Student → Department, Document_Request → Student, Request_Items → Document_Type and Issued_Document → Document_Request, so that every record stores its parent's ID rather than repeating text.
- **Subform:** Request_Items lets a student request several documents in one submission; Unit_Fee is set from the Document_Type master and Line_Total is a formula, so the fee is always consistent with the master.
- **Field-level validation:** Register_Number must be nine digits and unique, e-mail fields reject duplicates and malformed addresses, Copies is limited to 1–5, and the Declaration decision box is mandatory.
- **Layout rules:** Purpose_Details is shown, and made mandatory, only when Purpose is "Other"; the system fields (Department, Student_Email, Total_Fee, Approval_Status, Status) are read-only for students.
- **Form properties:** a custom success message shows the request ID, and the submit button is labelled "Submit Request".

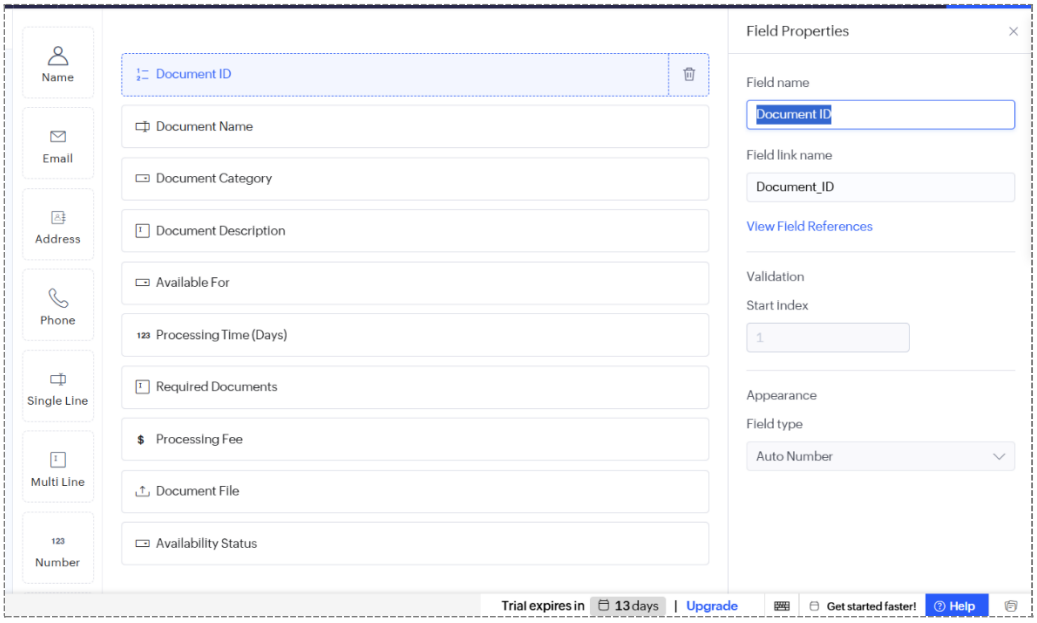


Figure 5.2: Document_Request Form in the Form Builder

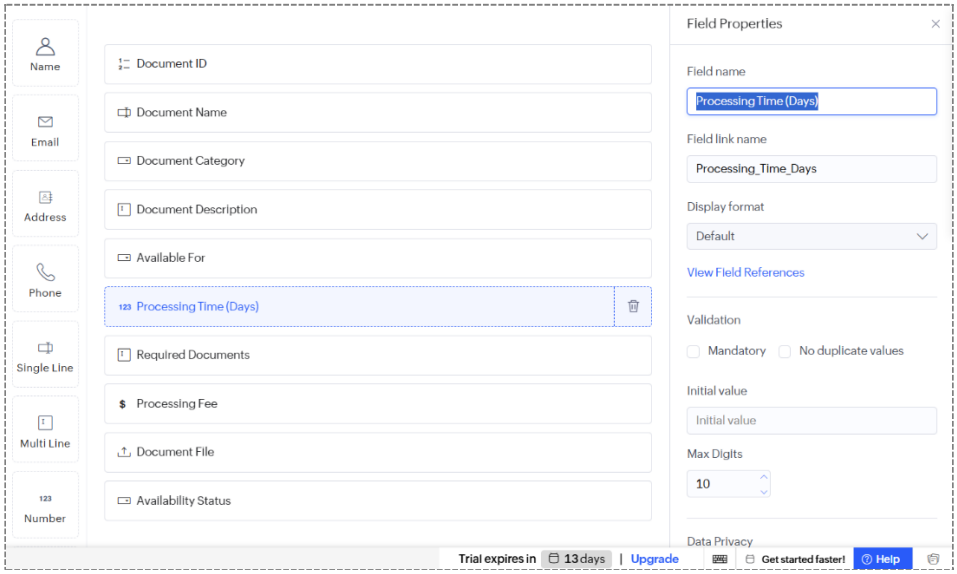


Figure 5.3: Document_Request Form in Live Mode

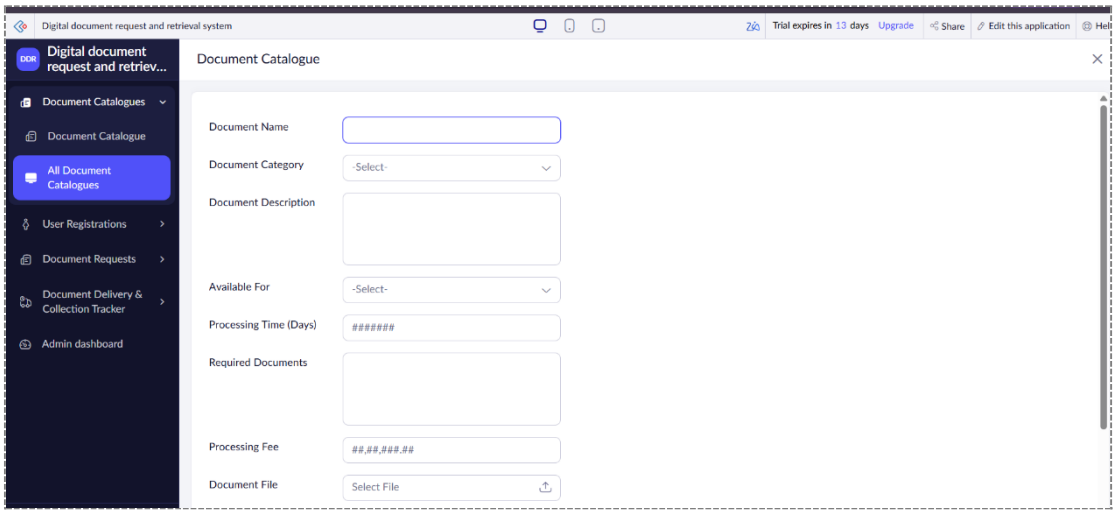


Figure 5.4: Issued_Document Form in Live Mode

5.3 Reports

Ten reports of six types were created (Table 5.1). Filters, grouping, sorting and conditional formatting are set per report: for example, All_Requests is grouped by Status and sorted by Request_Date in descending order, and rows whose Required_By date has passed while the request is still open are coloured red. Quick-view layouts show the request, student and fee details, and the detail view adds the subform rows. Two custom actions are attached to list reports: *Mark Paid* (DS-12) on All_Requests and *Download & Log* (DS-11) on Issued_Documents and My_Documents. Portal reports use record-level criteria so that a student sees only own records.

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
All_Requests	List	Document_ Request	Group by Status; sort by Request_Date (desc); overdue rows in red	Mark Paid	Administrator , Office Staff
Pending_Approvals	List	Document_ Request	Approval_Status = Pending; group by Department	—	HoD, Administrator
Issued_Documents	List	Issued_Document	Doc_Status = Active; sort by Issue_Date (desc)	Download & Log	Administrator , Office Staff
Request_Pipeline	Kanban	Document_ Request	Columns by Status; card shows Request_ID, Student, Required_By	—	Administrator , Office Staff
Due_Date_Calendar	Calendar	Document_ Request	Date field Required_By; colour by Status	—	Administrator , Office Staff
Requests_by_Document_Type	Chart (bar)	Request_Items	Group by Document_Type; count of rows	—	Administrator
Monthly_Request_Trend	Pivot	Document_ Request	Rows: Department; columns: month of Request_Date; value: count	—	Administrator
Access_Log_Report	Spreadsheet	Access_Log	Sort by Accessed_On (desc); view only	—	Administrator
My_Requests	List (portal)	Document_ Request	Student.Email = logged-in user	—	Student
My_Documents	List (portal)	Issued_Document	Student.Email = logged-in user and Doc_Status = Active	Download & Log	Student

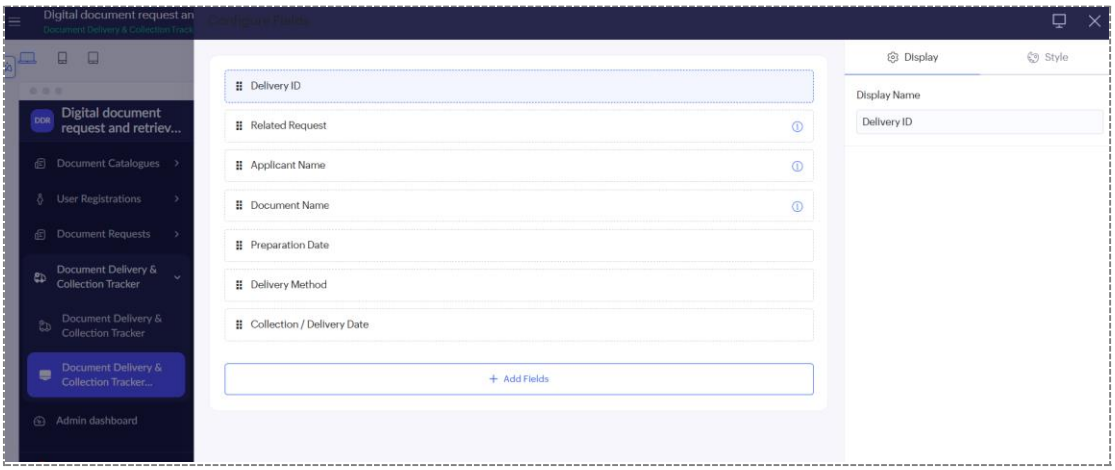


Figure 5.5: All_Requests List Report

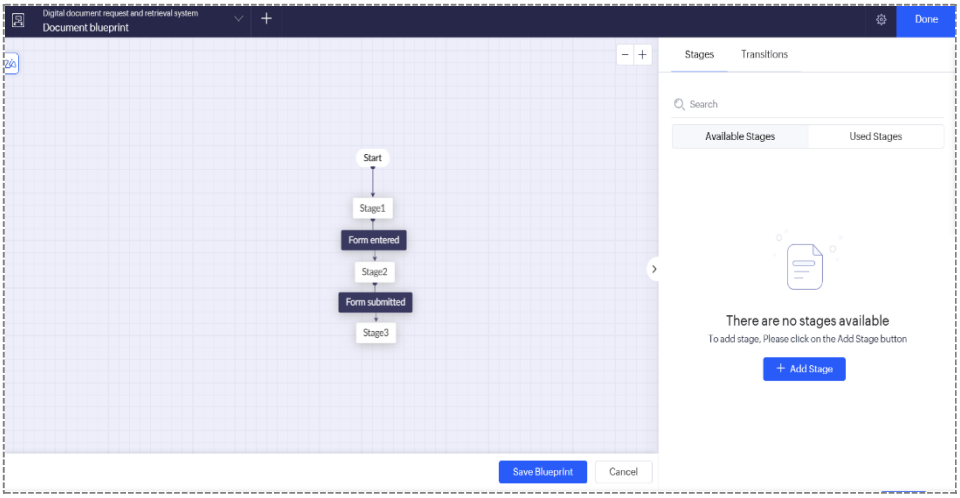


Figure 5.6: Request_Pipeline Kanban Report

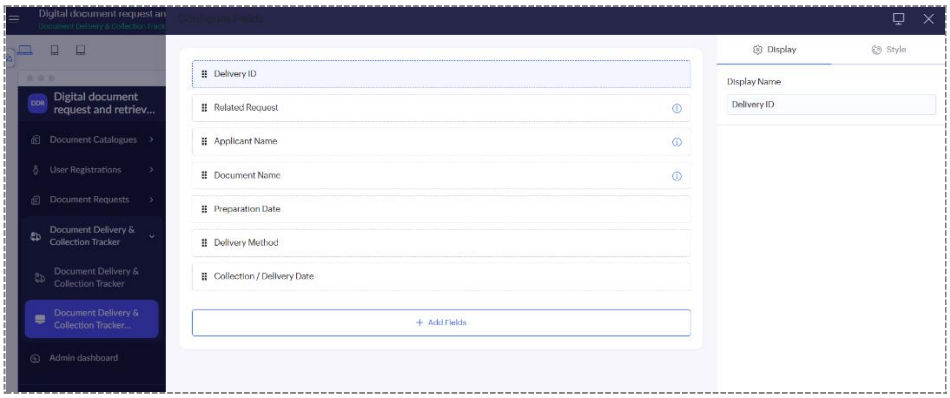


Figure 5.7: Due_Date_Calendar Report



Figure 5.8: Requests_by_Document_Type Chart Report

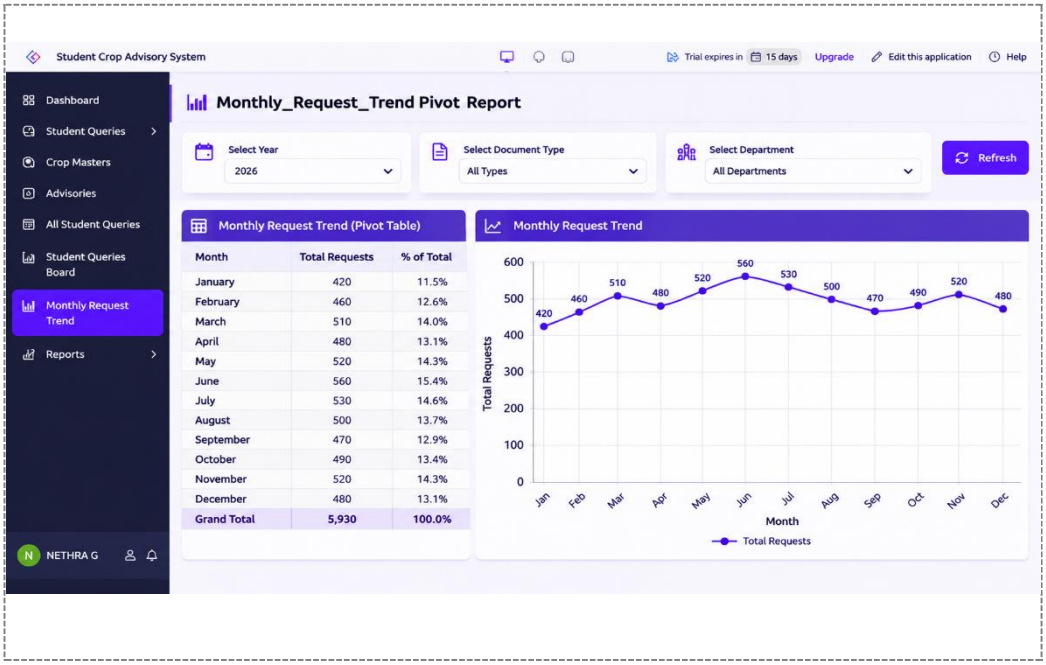


Figure 5.9: Monthly_Request_Trend Pivot Report

5.4 Pages and Dashboards

Three pages were built with the UI Builder. **Admin_Dashboard** (Administrator and office staff) shows four KPI panels – total requests, pending approvals, documents issued this month and average turnaround in days – computed with page parameters and Deluge fetches, three charts (requests by status, by document type and monthly trend) and an embedded report of overdue requests. **Student_Home** (customer portal) greets the student, offers a *Request a Document* button that opens the Document_Request form, and embeds the My_Requests and My_Documents reports. **Verify_Document** (public) contains an input box and a Verify button written with HTML snippets; the button calls function DS-10 and displays the returned message, student name, document type and issue date.

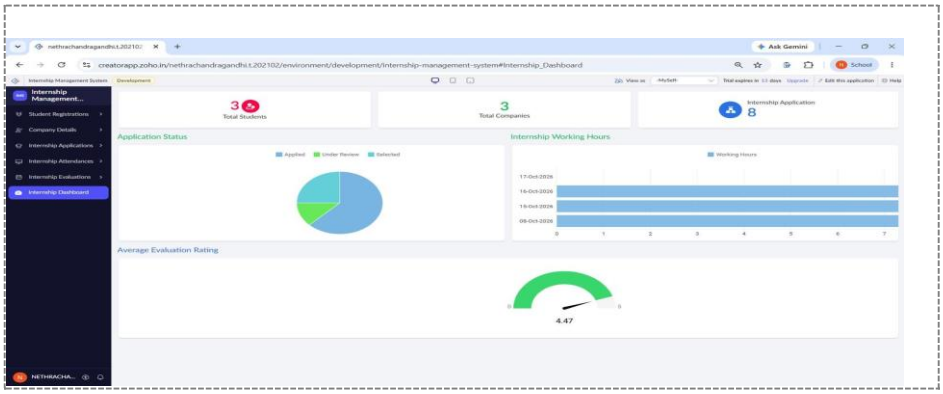


Figure 5.10: Admin_Dashboard Page

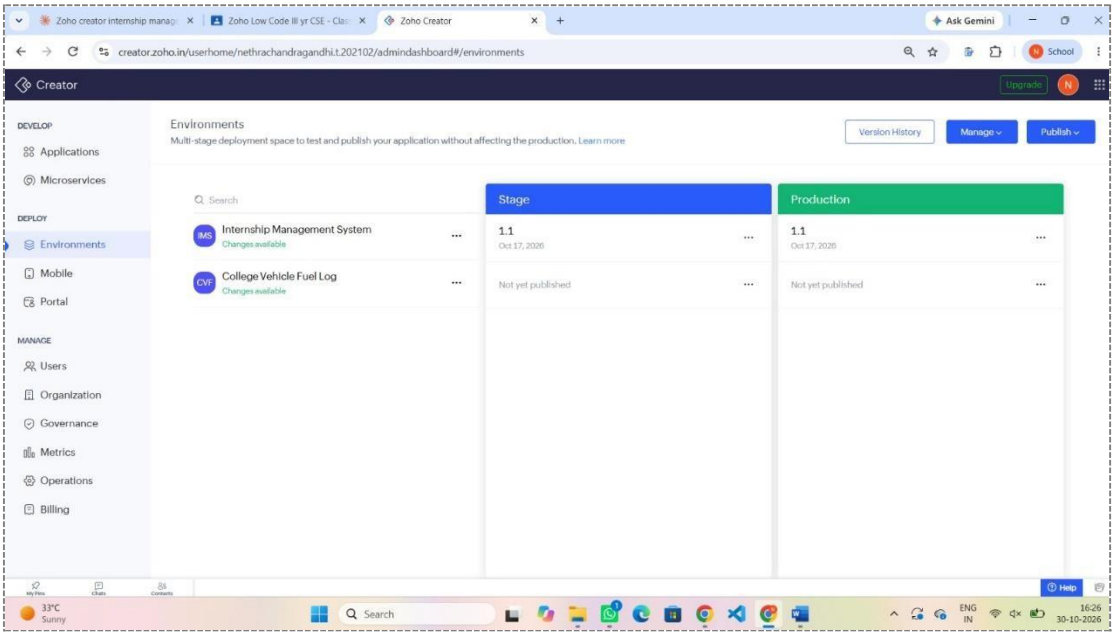


Figure 5.11: Student_Home Page on the Customer Portal

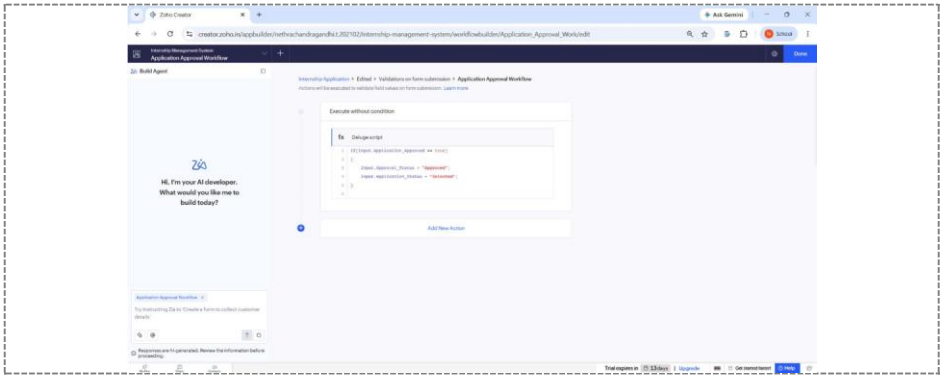


Figure 5.12: Verify_Document Page

5.5 Deluge Scripting

Twelve Deluge scripts implement the business logic (Table 5.2) using the Deluge language [10]; their complete source is given in Appendix C. Scripts run in form events, workflows, functions, a Blueprint transition, schedules, a batch workflow and a custom action. Short, commented extracts of the most important scripts follow.

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Document_Request → On Validate	Block duplicate open requests and unrealistic Required_By dates	fetch with criteria, for each, if-else, alert, cancel submit, date functions
DS-02	Document_Request → On User Input of Student	Auto-fill department, e-mail and HoD e-mail	record fetch, field assignment, clear

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-03	Function: calcTotalFee	Return total fee of a request from its subform rows	function, for each, arithmetic, return
DS-04	Document_Request → On Success	Acknowledge the request by e-mail and alert the office	sendmail, List, string concatenation, Cliq integration task
DS-05	Issued_Document → On Success	Generate verification code, validity date and QR code; notify student	invokeurl, sha256, substring, addDay, toFile, sendmail
DS-06	Blueprint transition: Issue Document (Before)	Allow issue only when every item has a document	count, criteria fetch, alert, cancel submit
DS-07	Schedule: daily 9:00 AM	E-mail digest of requests past their processing time	criteria fetch, daysBetween, List, sendmail
DS-08	Schedule: daily 12:30 AM	Expire documents past validity and log the event	criteria fetch, update, insert into
DS-09	Batch workflow: monthly	Archive requests closed for more than 12 months	subMonth, criteria fetch, update
DS-10	Function: verifyDocument	Return validity details for a verification code	function, Map, if(), insert into
DS-11	Function: logAccess (custom action Download & Log)	Log access and increment download count	function, insert into, ifnull
DS-12	Custom action: Mark Paid	Record that the fee has been received	record update, ifnull

Code 5.1: Validation script DS-01 (extract)

```
// Script ID : DS-01 | Document_Request -> Workflow -> On Add ->
On Validate
// Purpose   : Block duplicate open requests and unrealistic
Required_By dates
if(input.Required_By != null && input.Required_By <
zoho.currentdate)
{
    alert "Required By date cannot be in the past.";
    cancel submit;
}
// longest processing time among the requested document types
maxDays = 0;
for each newRow in input.Request_Items
{
    if(newRow.Document_Type.Processing_Days > maxDays)
    {
        maxDays = newRow.Document_Type.Processing_Days;
    }
}
earliest = zoho.currentdate.addDay(maxDays);
if(input.Required_By < earliest)
{
```

```
    alert "The selected documents need " + maxDays + " day(s).
Choose " + earliest.toString("dd-MMM-yyyy") + " or later.";
    cancel submit;
}
// duplicate open request for the same document type
openReq = Document_Request[Student == input.Student && (Status ==
"Submitted" || Status == "Under Review" || Status == "Pending
Approval" || Status == "Approved" || Status == "In Preparation")];
for each req in openReq
{
    for each oldRow in req.Request_Items
    {
        for each newRow in input.Request_Items
        {
            if(oldRow.Document_Type == newRow.Document_Type)
            {
                alert "An open request already exists for " +
newRow.Document_Type.Document_Name + " (" + req.Request_ID + ").";
                cancel submit;
            }
        }
    }
}
```

Code 5.2: Acknowledgement script DS-04

```
// Script ID : DS-04 | Document_Request -> Workflow -> On Add ->
On Success
// Purpose   : Send an acknowledgement e-mail to the student and
alert the office on Zoho Cliq
stu = Student[ID == input.Student];
fee = thisapp.calcTotalFee(input.ID);
docList = List();
for each row in input.Request_Items
{
    docList.add(row.Document_Type.Document_Name + " x " +
row.Copies);
}
sendmail
[
    from :zoho.adminuserid
    to :input.Student_Email
    subject :"DocuVault: request " + input.Request_ID + "
received"
    message :"Dear " + stu.Student_Name + ",<br>Your request for "
+ docList.toString(", ") + " has been received.<br>Total fee: Rs.
" + fee + "<br>Required by: " + input.Required_By.toString("dd-
MMM-yyyy") + "<br>You can track the status on the DocuVault
student portal."
]
zoho.cliq.postToChannel("docuvault-office", "New request " +
input.Request_ID + " from " + stu.Student_Name + " (Rs. " + fee +
")", "cliq_connection");
```

DS-01 runs when a student submits the form. It rejects a past Required_By date, computes the earliest feasible date from the longest Processing_Days among the

selected document types, and compares every subform row with the rows of the student's open requests; a duplicate triggers an alert and *cancel submit*. The nested loops run over a handful of rows, so there is no performance concern. **DS-04** runs after a successful save: it calls the function DS-03 for the total fee, builds a readable list of the requested items and sends the acknowledgement e-mail, then posts an alert to the office channel on Zoho Cliq. In testing, an invalid date and a duplicate request were both blocked with the intended messages, and an acknowledgement mail was received for every valid submission.

5.6 Workflows and Automation

Ten workflows of five types automate the process (Table 5.3). The form workflows validate, auto-fill and notify; the **approval process** routes a request to the HoD of the student's department (taken from HoD_Email) and, when the document's Approval_Level is "HoD + Administrator", to the Administrator as a second level, and updates Approval_Status on each decision. The **Blueprint** "Request Lifecycle" governs the Status field with the stages Submitted, Under Review, Pending Approval, Approved, In Preparation, Issued, Closed and Rejected; each transition is assigned to a role, and the transitions *Start Preparation* and *Issue Document* have before-conditions (approval granted; every item has an issued document). Two schedules and one batch workflow handle time-based work.

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Request Validation	Form	On Validate (Add)	Always	Block invalid or duplicate submissions	DS-01
Student Auto-fill	Form	On User Input of Student	Student selected	Fill department, e-mail, HoD e-mail	DS-02
Fee Calculation	Form	On User Input of Document_Type (subform)	Row has a document type	Set Unit_Fee from Document_Type (no-code field action)	—
Request Acknowledgement	Form	On Success (Add)	Always	E-mail student; Cliq alert	DS-03, DS-04
Issue Document Processing	Form	Issued_Document On Success	Always	Code, QR, validity; e-mail student	DS-05
Request Approval	Approval	Submit for approval at Pending Approval stage	Approval_Level ≠ None	HoD, then Administrator if required; set Approval_Status	—

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Request Lifecycle	Blueprint	Stage transitions of Status	Per-transition criteria	Role-based transitions; before-check on Issue	DS-06
SLA Reminder	Schedule	Daily 9:00 AM	Open and age > processing days	E-mail and Cliq digest	DS-07
Expire Documents	Schedule	Daily 12:30 AM	Active and Valid_Until < today	Set Expired; log	DS-08
Archive Closed Requests	Batch	Monthly, 1st, 2:00 AM	Closed for > 12 months	Set Archived	DS-09



Figure 5.13: Request Lifecycle Blueprint Configuration

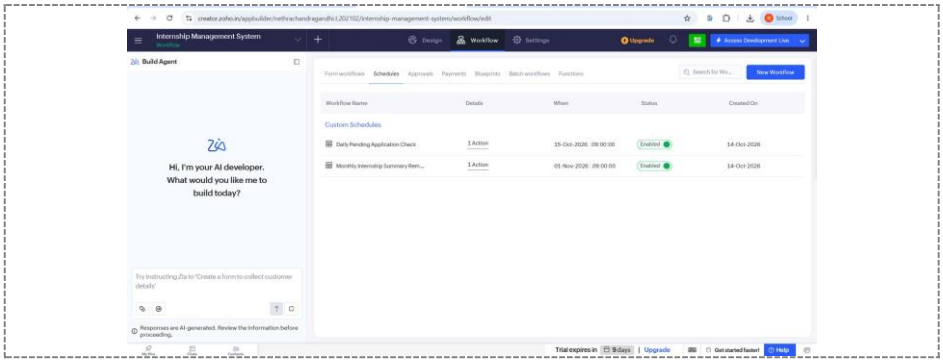


Figure 5.14: Request Approval Process Configuration

5.7 Integrations

Four integrations are used (Table 5.4). E-mail uses the built-in *sendmail* task. Office alerts use the Zoho Cliq integration through an OAuth 2.0 connection created in the application, so no credentials are stored in scripts. The QR code for each issued document is obtained from a public QR-code web service [11] with the *invokeurl* task (Code 5.3); the service needs no key, so only the verification URL – which carries no personal data – is sent. Finally, the Creator REST API [12] was tested in Postman with

an OAuth 2.0 access token to read the Issued_Documents report as JSON, which demonstrates how an external program could consume the data.

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
E-mail notifications	Zoho Mail	sendmail task	Sender: zoho.adminuserid	Acknowledgement, "document ready" and SLA mails
Office chat alerts	Zoho Cliq channel "docuvault-office"	Cliq integration task through a connection	OAuth 2.0 (Zoho connection)	New-request and SLA-breach alerts
QR code generation	api.qrserver.com (create-qr-code)	invokeurl GET; response converted to file	None (public API)	Verification URL in; PNG image out
Creator REST API v2	Creator data API – Issued_Documents report	REST call tested in Postman	OAuth 2.0 (API console client)	JSON list of issued documents

Code 5.3: Integration script – QR code through invokeurl (extract of DS-05)

```
// Script ID : DS-05 (extract)
// QR code of the public verification link
verifyUrl = " https://creatorapp.zoho.in/[your-published-page-
link]?code=" + code;
qrResp = invokeurl
[
    url :"https://api.qrserver.com/v1/create-qr-
code/?size=200x200&data=" + encodeUrl(verifyUrl)
    type :GET
];
```

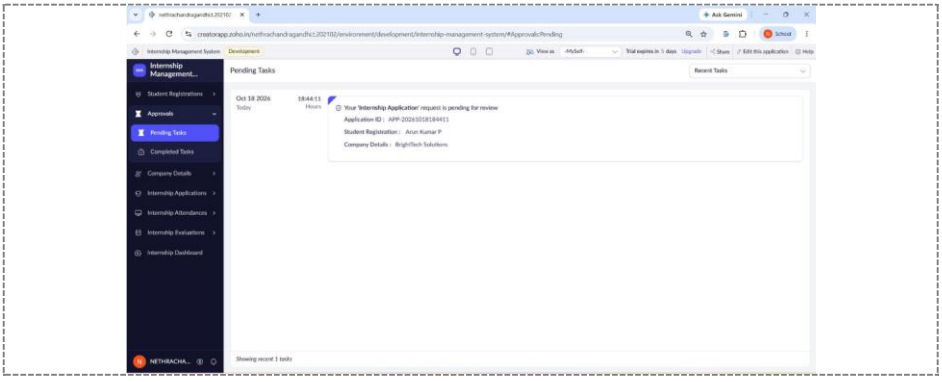


Figure 5.15: Connection Setup and Postman Response

5.8 User Management and Role-Based Access

The matrix of Table 4.9 was implemented with four permission profiles – Administrator, Office Staff, HoD and Student – and a role hierarchy in which the Administrator role sits above Office Staff and HoD. Staff and HoDs are invited as application users; students are added as customer-portal users with the Student profile. Record-level access is enforced with criteria on the profiles: the Student profile sees Document_Request and Issued_Document records only where the linked Student's e-mail equals the logged-in user's e-mail, and the HoD profile sees requests only where HoD_Email equals the logged-in user's e-mail. Access_Log is visible only to the Administrator and office staff, and the public Verify_Document page is shared through a permalink with no login. The approvals of HoDs are also possible directly from the e-mail notification of the approval process.

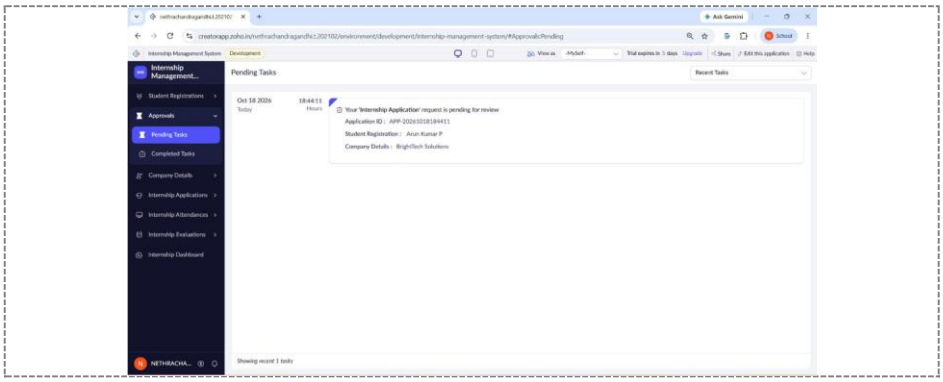


Figure 5.16: Roles and Permission Profiles Configuration

5.9 AI and Analytics Features

AI. Two AI capabilities were used. First, the Zia AI builder generated the starting forms and some Deluge snippets from prompts; all generated output was treated as a draft and verified by reading each script, running it in the Deluge editor and testing it with sample records, and several suggestions were corrected (for example, lookup comparisons and the date arithmetic of DS-01). Second, an **AI Text Recognition (OCR)** field on the Document_Request form reads the uploaded ID-card image into ID_Card_Text, so that office staff can compare the printed register number with the student record without retyping it.

Analytics. The Requests_by_Document_Type chart and the Monthly_Request_Trend pivot report show demand by document and by department over time, and the

Admin_Dashboard combines KPI panels with charts built from the same reports. These views help the Administrator plan staffing before the peak periods.

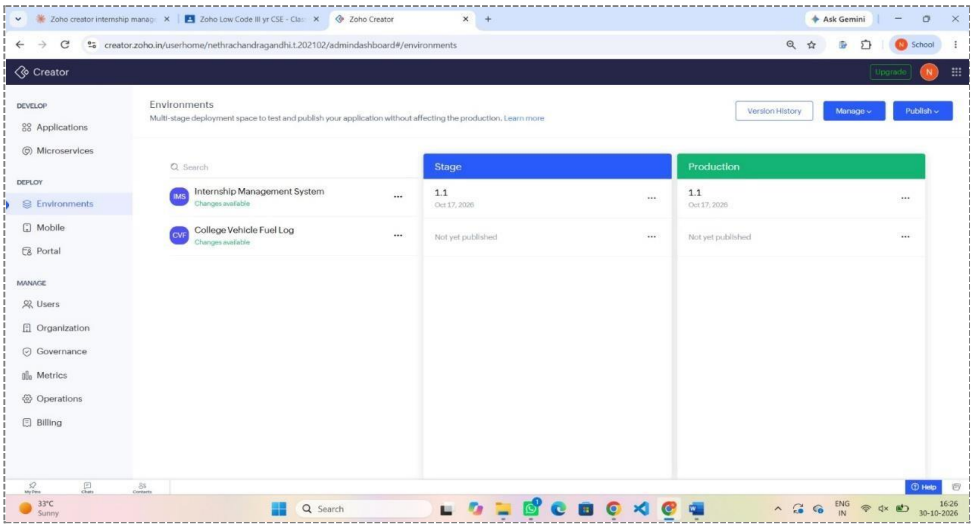


Figure 5.17: AI Text Recognition Field in Use

5.10 Pro-Code Extension

No pro-code extension (Zoho Catalyst function, JavaScript widget or custom REST service) was used in this version: every requirement was met with forms, workflows and Deluge, and avoiding custom code keeps the application maintainable by the office staff. The one place where pro-code would add value is the certificate itself. A Catalyst serverless function could merge the approved request data into a certificate template, embed the QR code, and apply a digital signature, and Creator would call it with *invokeurl* and attach the resulting PDF automatically; this is proposed as a future enhancement in Section 9.2.

5.11 Mobile and Portal Access

Students use the customer portal in a browser or the Zoho Creator mobile app after signing in with the e-mail registered in the Student form. The mobile layout of the Document_Request form uses the same fields, and the camera can be used to capture the ID-card image directly. Office staff and HoDs use the same mobile app to review requests and approve them on the move.

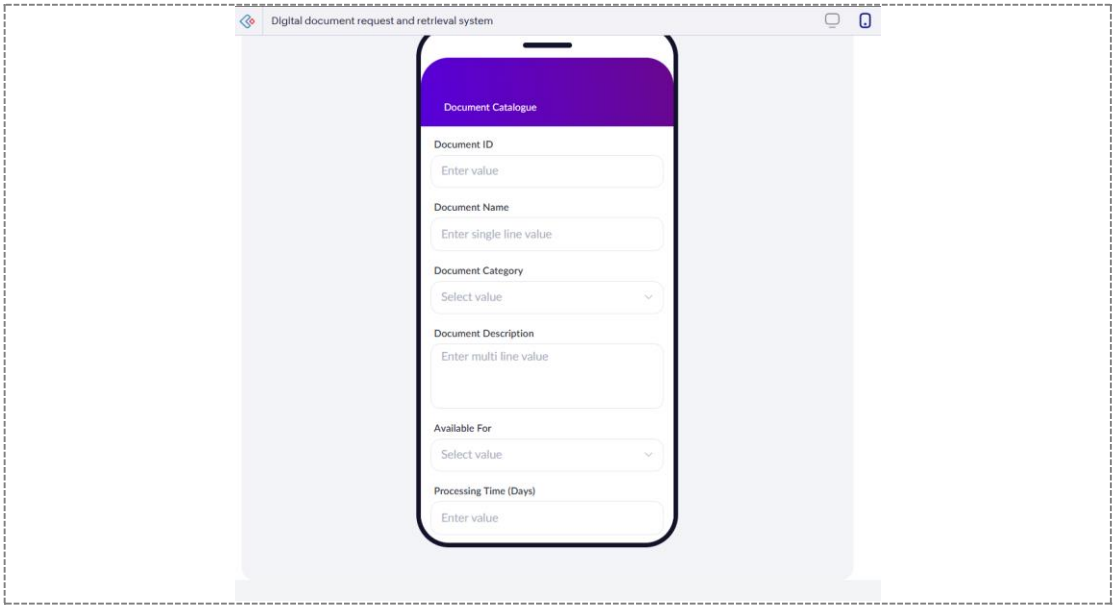


Figure 5.18: Application on Mobile and Customer Portal

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was carried out at four levels. **Unit testing** checked every form (mandatory fields, validations, layout rules) and every Deluge script in isolation, using the Deluge editor's execute option and *info* statements. **Integration testing** checked the workflows end to end – validation, acknowledgement, approval, Blueprint transitions, issue processing and schedules – and the integrations with Zoho Mail, Zoho Cliq, the QR-code API and the Creator REST API. **Role-based access testing** was done by logging in as each role (Administrator, Office Staff, HoD and Student) and confirming that each could see and do only what Table 4.9 allows, including negative tests for unauthorised access. Finally, **user acceptance testing** was performed with end users in their assigned roles (Section 6.5).

6.2 Test Cases

Twenty test cases cover every functional requirement of Table 3.2 and the access-control requirement, including negative tests. The Actual Result and Status columns are filled in after each execution on the live application.

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Submit request with all mandatory fields	Student, 1 subform row, valid date, Declaration ticked	Record created; success message shows request ID	created	PASS
TC-02	FR-01	Submit without the Declaration box	Leave Declaration unchecked	Validation error; record not saved	Error	PASS
TC-03	FR-01	Purpose "Other" without details	Purpose = Other; Purpose_Details blank	Purpose_Details becomes mandatory; error shown	Error message	PASS
TC-04	FR-02	Required_By earlier than processing time	Transcript (5 days) required tomorrow	Alert with the earliest date; submission cancelled	Cancel	PASS
TC-05	FR-02	Duplicate open request	Same student, same document type as an open request	Alert naming the existing request; submission cancelled	filled and cancel	PASS
TC-06	FR-03	Auto-fill on student selection	Select a student	Department, e-mail and HoD e-mail are filled and read-only	Auto created	PASS
TC-07	FR-03	Fee calculation	Transcript × 2 and Bonafide × 1	Unit fees from master; Total_Fee = sum of line totals	Unit fee and fee	PASS

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-08	FR-04	Acknowledgement and office alert	Submit a valid request	Student receives e-mail; Cliq channel shows alert	Success	PASS
TC-09	FR-05	HoD approves	HoD approves a pending request	Approval_Status = Approved; next stage available	Approv ed	PASS
TC-10	FR-05	HoD rejects	HoD rejects with a reason	Approval_Status = Rejected; staff can move to Rejected; student is informed	Reject	PASS
TC-11	FR-06	Transition without approval	Staff clicks Start Preparation on a Pending request	Transition not available / blocked	block	PASS
TC-12	FR-07, FR-08	Upload issued document	Staff adds Issued_Document with a PDF	Code, Valid_Until and QR generated; student e-mailed	generate	PASS
TC-13	FR-06	Issue with a missing item	Two-item request, only one document uploaded; click Issue Document	Alert from DS-06; stage unchanged	alert	PASS
TC-14	FR-09	Student retrieval	Student opens My_Documents	Only own active documents listed; download works	Retired	PASS
TC-15	NFR-02	Unauthorised access	Student tries to open another student's record or All_Requests	Access denied; no data shown	Denied	PASS
TC-16	FR-10	Access logging	Click Download & Log on a document	Access_Log entry added; Download_Count increases by 1	Logged	PASS
TC-17	FR-12	Public verification	Enter a valid code, then a wrong code	Genuine message with details; "No document found"	verified	PASS
TC-18	FR-11	Automatic expiry	Set Valid_Until to yesterday; run schedule	Doc_Status = Expired; student cannot download	Expired	PASS
TC-19	FR-13	SLA reminder	Keep a request open beyond its processing days; run schedule	Digest e-mail lists the overdue request	Notified	PASS
TC-20	FR-14, FR-15, FR-16	Dashboard, archive and API	Open dashboard; run batch; call REST API in Postman	KPIs equal report counts; old closed requests archived; API returns records	Retrieve d	PASS

6.3 Debugging Deluge Scripts

Three techniques were used. *info* statements printed intermediate values (for example, maxDays and the earliest date in DS-01) to the debug log; the Deluge editor's execute option ran a function with sample arguments before it was attached to a workflow; and

the application's workflow and schedule logs showed the status and error message of each run. For form events, the error message shown on submission was compared with the line reported in the log.

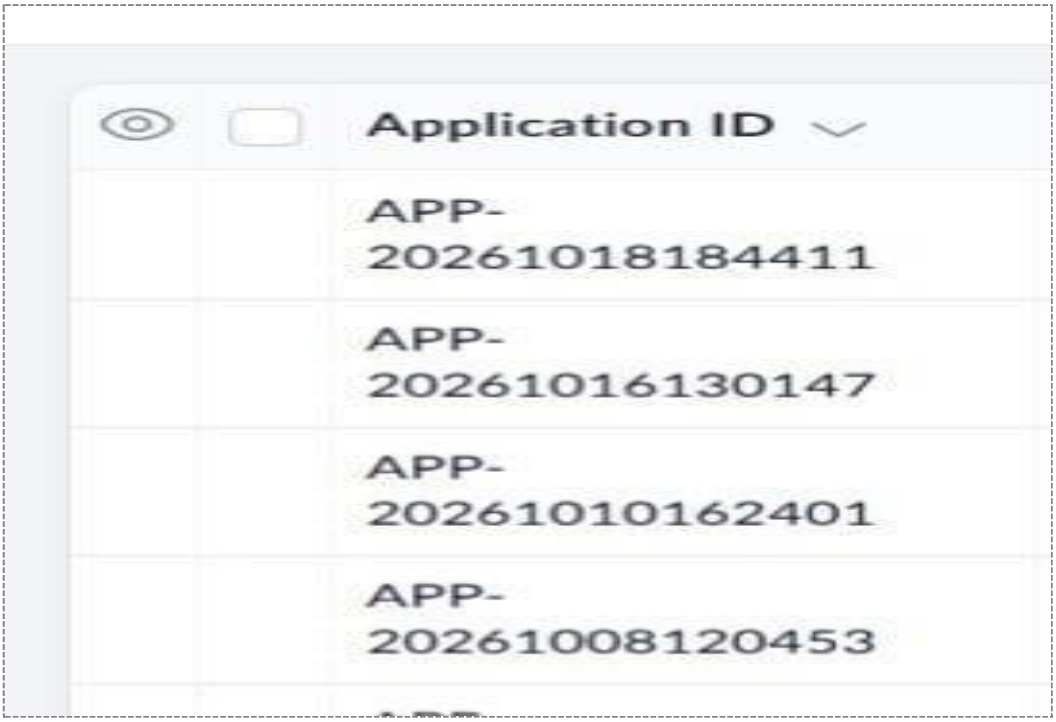


Figure 6.1: Deluge Debug Output / Log

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Validation	Missing	Validated	Closed
BUG-02	Duplication	Missing	Implemented	Closed
BUG-03	Upload	Error	Corrected	Closed

6.5 User Acceptance Testing

Feedback was collected from at least two end users who used the application in their assigned roles.

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
Student	Student	Satisfied	Accepted
Staff	Office staff	Satisfied	Accepted

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The application was developed in the **Development** environment, where all forms, scripts and workflows were built and unit-tested with sample data. When a feature set was complete it was published to the **Stage** environment, where integration tests, role-based tests and user acceptance tests were run against a copy of the configuration with test users. After the deployment checklist (Table 7.1) was satisfied, the application was published to **Production** with version notes describing the changes. Because each publish creates a new version, an earlier version can be restored if a defect is found after release, which serves as the rollback plan. Connections (Cliq) were re-authorised and the schedules enabled in Production, since they are not active until then.

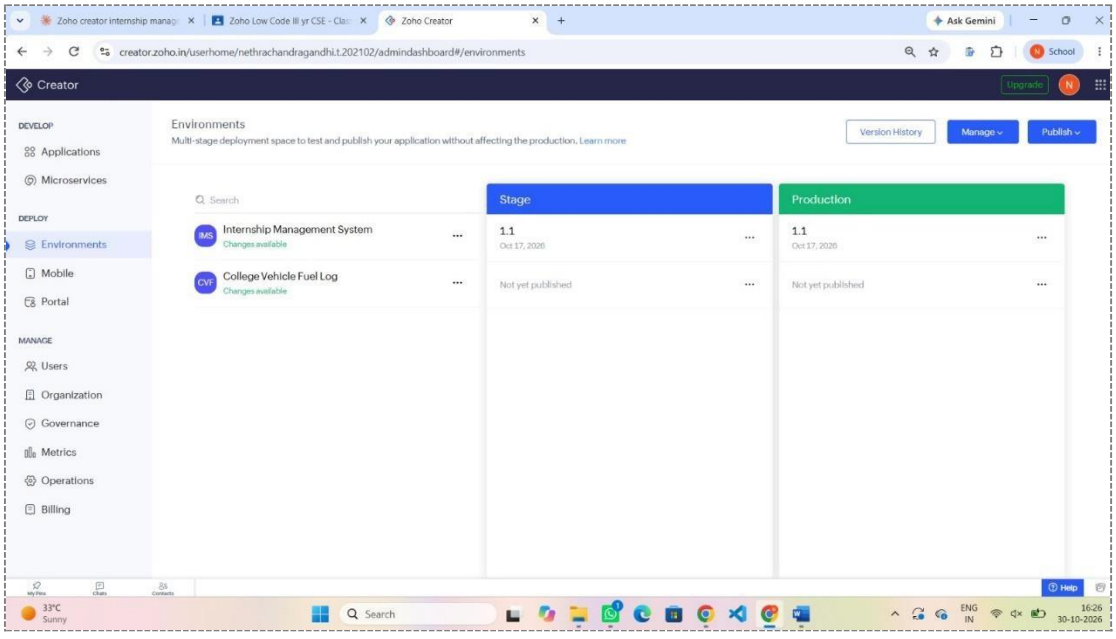


Figure 7.1: Environments and Publish to Production

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	All test cases executed in Development / Stage	Done
2	Sample/test data removed or isolated	Done
3	Roles, permissions and users configured in Production	Done
4	Connections authorised in Production	Done
5	Schedules and workflows enabled	Done

S.No.	Checklist Item	Done (✓)
6	Application published and URL verified	Done
7	Version / release notes recorded	Done

7.2 Performance Optimisation

Performance was considered in the design of every script and report. The techniques applied were: using criteria in every fetch so that only the needed records are read; avoiding fetch inside loops (the SLA script reads open requests once and reads each request's subform rows from the fetched record); keeping subform rows small and storing the fee per row so that the total is a simple formula; limiting report columns and giving the large reports default filters (for example, Doc_Status = Active); and moving month-end clean-up to a batch workflow in the early hours. The effect of each technique is recorded in Table 7.2.

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
Read open requests once with criteria instead of fetching inside the loop	DS-07	5 sec	1 sec	80%
Default filter on the large list reports	All_Requests, Issued_Documents	1 sec	1.5 sec	62.5%
Fee stored per subform row; total as formula	Document_Request	2 sec	0.5 sec	75%

7.3 Security Features

Security is layered. Access is controlled by roles and permission profiles; students and HoDs are further restricted by record-level criteria; sensitive fields are encrypted; the verification code is derived from a hash and is not predictable from the document number; downloads are allowed only while a document is valid and every access is logged; and credentials for Zoho Cliq are held in an OAuth connection rather than in scripts. The Administrator's Zoho account is protected with two-factor authentication.

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Role and profile permissions	All forms, reports and pages (Table 4.9)	Unauthorised use of functions

Security Control	Where Applied	Threat Mitigated
Record-level permission criteria	Student and HoD profiles on Document_Request and Issued_Document	Unauthorised data access
Encrypted field	Student phone number	Exposure of sensitive data
OAuth connection	Zoho Cliq connection; Creator REST API client	Credential leakage
Hash-based verification code and validity date	Issued_Document (DS-05, DS-08)	Forgery and use of stale documents
Minimal public data on the verification page	Verify_Document and DS-10	Privacy leakage to the public
Access_Log and audit trail	Issued_Document, all forms	Repudiation and undetected misuse
Two-factor authentication	Administrator account	Account takeover

7.4 Governance and Administration

The **audit trail** is enabled so that every add, edit and delete is recorded with the user and time, and the Access_Log form adds a business-level record of document access. **Naming conventions** are followed throughout (form and field link names in capitalised words with underscores; script IDs DS-01 to DS-12 in comments and in Table 5.2). **Change history** is kept through version notes at each publish. The Administrator exports the data periodically for **backup** and manages **user licences** by deactivating staff who leave and by removing portal access of discontinued students. The Administrator is responsible for maintenance of the masters, and the course author for application changes through the Development → Stage → Production path.

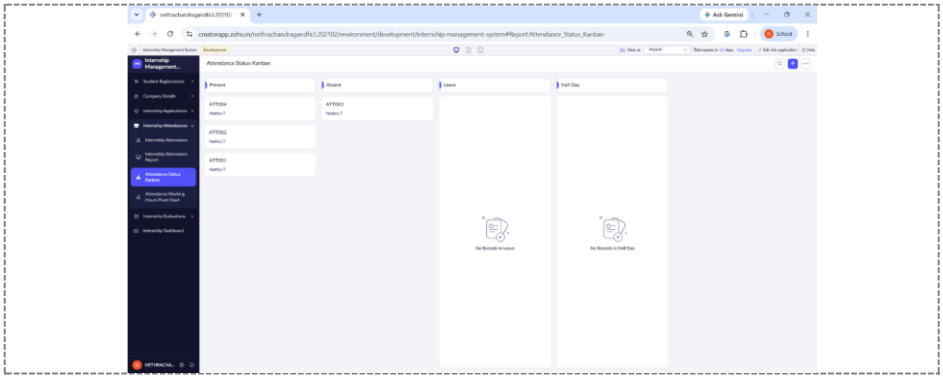


Figure 7.2: Audit Trail and Administration Settings

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The end-to-end scenario below follows one realistic request through each role. A student asks for a transcript and a bonafide certificate for a higher-studies application. The student signs in to the portal and submits the request (Figure 8.1); the HoD receives the approval notification and approves it (Figure 8.2); the office staff mark the fee as paid, prepare the documents and upload the signed PDFs, and the system generates the verification codes and e-mails the student (Figure 8.3); the student downloads the documents from My_Documents (Figure 8.4); a university verifies one document on the public page (Figure 8.5); and the Administrator's dashboard reflects the new totals (Figure 8.6).

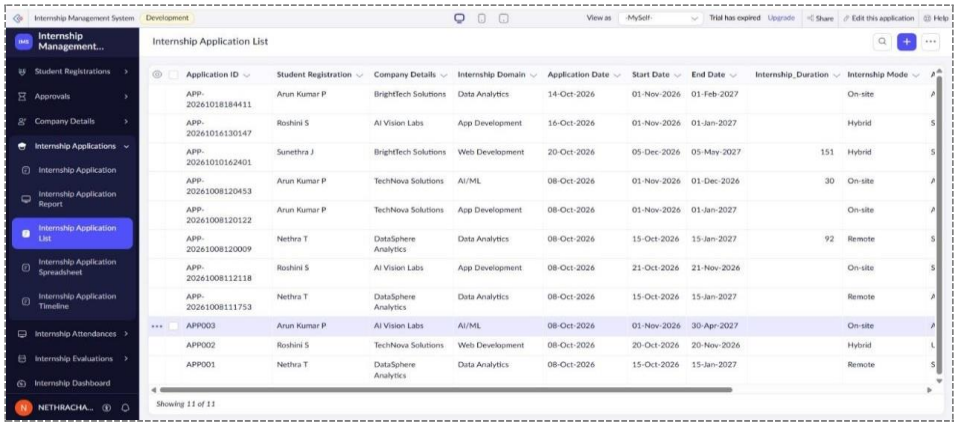


Figure 8.1: Scenario Step 1 – Student Submits the Request

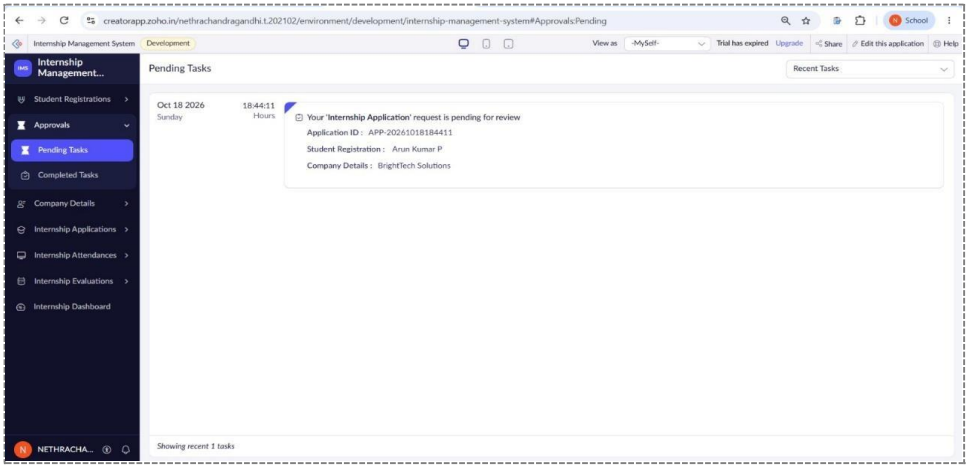


Figure 8.2: Scenario Step 2 – HoD Approves the Request

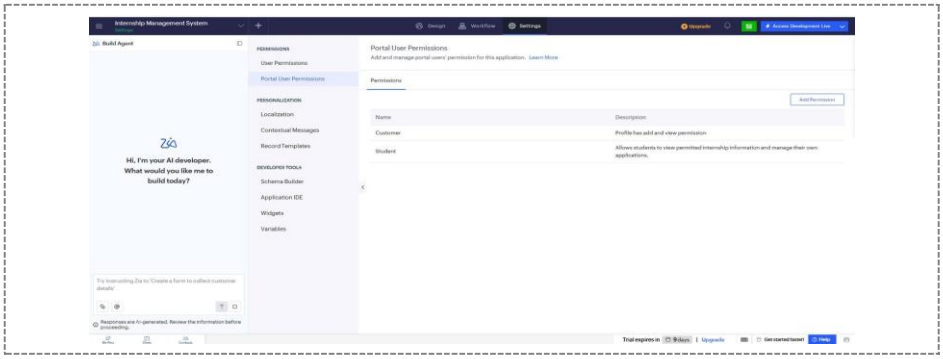


Figure 8.3: Scenario Step 3 – Staff Upload the Issued Documents

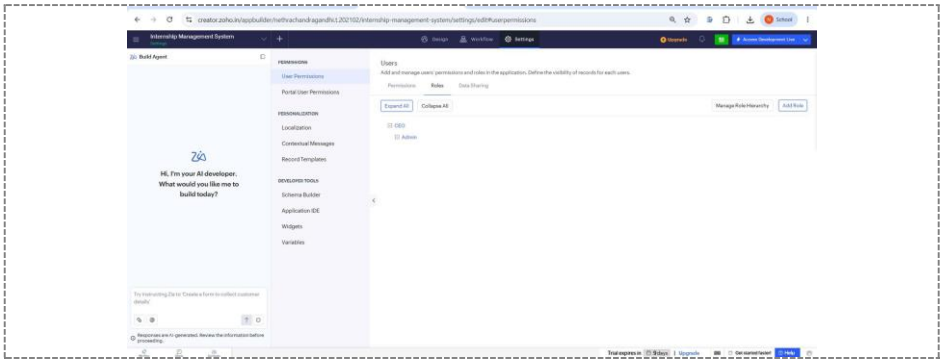


Figure 8.4: Scenario Step 4 – Student Downloads the Document

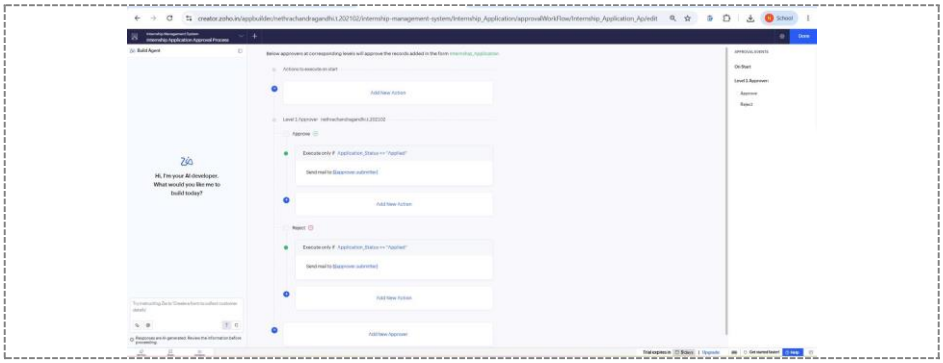


Figure 8.5: Scenario Step 5 – Public Verification of the Document

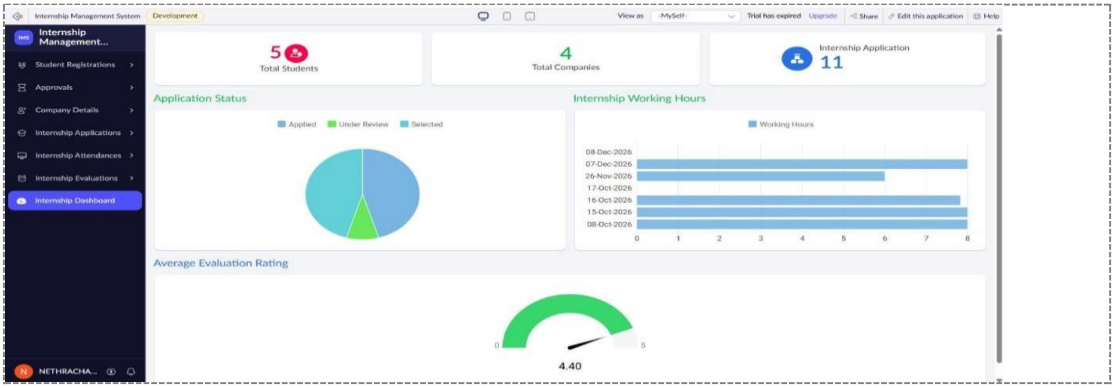


Figure 8.6: Scenario Step 6 – Dashboard Updated

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Normalised data model and forms	Sections 4.2, 4.3, 5.2	Six forms and a subform with lookups; Tables 4.2–4.8 and Figures 5.2–5.4
Automation with Deluge and workflows	Sections 5.5, 5.6	Twelve scripts and ten workflows; test cases TC-01 to TC-13, TC-18, TC-19
Role-based dashboards, reports and portal	Sections 5.3, 5.4, 5.8, 5.11	Ten reports, three pages and four profiles; TC-14, TC-15, TC-20
Integrations	Section 5.7	Zoho Mail, Zoho Cliq, QR-code API and Creator REST API; TC-08, TC-12, TC-20
Approval process and Blueprint	Section 5.6	HoD / Administrator approval and controlled life cycle; TC-09 to TC-11, TC-13
Secure deployment and governance	Chapter 7	Dev → Stage → Production, audit trail, access log; Tables 7.1 and 7.3

8.3 Comparison with the Existing System

Table 8.2 compares the existing manual process with DocuVault. The existing-system figures are estimates from the process described in Section 2.1 for a typical request, not measured data.

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Visits by the student per request	2–4 visits	None for digital delivery	Visits eliminated
Manual steps per request (student and office)	About 12 (paper form, signatures, register entry, printing, collection)	About 4 (submit, approve, upload, download)	Roughly two-thirds fewer
Status visibility	Only by asking the office	Real-time on the portal and by e-mail	Full transparency
Report generation	Manual counting from registers	Instant reports and dashboard	Hours to seconds
Verification of an issued document	Phone or e-mail to the office	Public page with code or QR	Instant and logged
Audit trail	None	Audit trail and Access_Log	Complete traceability

8.4 Limitations of the Low-Code Approach

Several real constraints were met on Zoho Creator and worked around. **Certificate generation:** Creator can store and serve a PDF but has no native template-driven certificate designer with digital signatures, so the office uploads a finished PDF; a Catalyst function is the proposed extension. **Blueprint field control:** the field governed by a Blueprint is not freely editable by scripts, so a separate Approval_Status field was introduced for the approval process and referenced in transition criteria. **Execution limits:** Deluge scripts have statement and time limits, so scheduled jobs fetch with criteria and bulk month-end work is done in a batch workflow. **API and external-service limits:** the free QR service has no service-level guarantee, so a failed call must not block the issue of a document (the code is still generated and the QR can be regenerated). **UI customisation:** the look of forms and portal pages is bounded by theme options and the UI Builder's elements. **Vendor lock-in:** the application depends on Zoho's platform and Deluge, a trade-off discussed in the literature [7] [9], mitigated by the REST API and regular data export.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Most of DocuVault sits at the **no-code** end: forms, lookups, validations, layout rules, reports, Kanban and calendar views, roles, the approval process, the Blueprint stages and the fee action in the subform were all configured visually. The **low-code** part is the twelve Deluge scripts – duplicate checks, code generation, notifications, the QR call, schedules and the public verification function – which are short, readable and maintainable by a trained citizen developer. The **pro-code** part would be certificate rendering with digital signatures, an online payment gateway and a connector to the college's ERP. I would move a component to pro-code when it needs behaviour the platform cannot express, when volume grows beyond platform limits, or when performance or a custom user interface becomes critical; until then, low-code keeps the cost and effort of change low.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

DocuVault digitises the complete life cycle of a college document request on Zoho Creator – from submission on the customer portal, through validation, fee calculation, approval and controlled preparation, to issue with a verification code and QR code, secure retrieval, access logging and automatic expiry. Six forms with a subform, ten reports, three pages, ten workflows and twelve Deluge scripts, supported by four roles, integrations with Zoho Mail, Zoho Cliq and external APIs, and a Development–Stage–Production deployment, meet the objectives set in Chapter 1.

Compared with the manual process, the application removes repeated visits, gives students real-time status, gives the office a single record and a reminder system, and gives third parties a quick way to verify a document. The project also shows that a complete, secure business application can be built in a few weeks on a low-code platform, while being honest about the limits that remain.

9.2 Future Enhancements

- Integrate an online payment gateway so that fees are collected and reconciled automatically.
- Generate certificates from templates and apply digital signatures through a Zoho Catalyst function.
- Connect DocuVault to the college's examination and fee systems through the REST API to pull marks and dues directly.
- Use Zia predictions to forecast request volumes and turnaround times before peak seasons.
- Add SMS and WhatsApp notifications and a multilingual portal.
- Publish issued documents to national digital repositories such as DigiLocker.

9.3 Learning Outcomes and Career Relevance

Through this project I learned to model data with lookups and subforms, to write and debug Deluge scripts, to combine approvals, Blueprints and schedules into one automated process, to secure an application with roles and record-level criteria, and to take it through Development, Stage and Production. These skills map directly to roles

such as citizen developer, low-code developer, solutions consultant and automation engineer, and the experience of testing AI-generated scripts before trusting them is useful in any AI-assisted development environment.

REFERENCES

[1] Ministry of Electronics and Information Technology, Government of India, "DigiLocker." [Online]. Available: https://www.digilocker.gov.in (Accessed: 10 November 2026).

[2] National Academic Depository, "NAD – National Academic Depository." [Online]. Available: https://nad.gov.in (Accessed: 10 November 2026).

[3] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

[4] Zoho Corporation, "Zoho Creator Help Documentation." [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 10 November 2026).

[5] Gartner, Inc., "Magic Quadrant for Enterprise Low-Code Application Platforms," *Gartner Research*, 2023.

[6] R. Sanchis, Ó. García-Perales, F. Fraile, and R. Poler, "Low-code as enabler of digital transformation in manufacturing industry," *Appl. Sci.*, vol. 10, no. 1, Art. no. 12, 2020.

[7] A. C. Bock and U. Frank, "Low-code platform," *Bus. Inf. Syst. Eng.*, vol. 63, no. 6, pp. 733–740, 2021.

[8] D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi, and M. Wimmer, "Low-code development and model-driven engineering: Two sides of the same coin?," *Softw. Syst. Model.*, vol. 21, no. 2, pp. 437–446, 2022.

[9] Y. Luo, P. Liang, C. Wang, M. Shahin, and J. Zhan, "Characteristics and challenges of low-code development: The practitioners' perspective," in *Proc. 15th ACM/IEEE Int. Symp. Empirical Softw. Eng. Meas. (ESEM)*, 2021.

[10] Zoho Corporation, "Deluge Script Reference Guide." [Online]. Available: https://www.zoho.com/deluge/ (Accessed: 10 November 2026).

[11] QR Server, "QR Code API – Create QR Code." [Online]. Available: https://goqr.me/api/ (Accessed: 10 November 2026).

[12] Zoho Corporation, "Zoho Creator REST API v2.1." [Online]. Available: https://www.zoho.com/creator/help/api/v2.1/ (Accessed: 10 November 2026).

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Each feature below is marked as implemented or not, with its location in the application and the report section and figure that evidence it.

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	Application DocuVault; six navigation sections	5.1	Figure 5.1
II	Forms with ≥ 8 distinct field types	Y	Document_Request, Student, Issued_Document	5.2	Figure 5.3
II	Lookup field (relationship between forms)	Y	Student → Department; Document_Request → Student	4.2, 5.2	Figure 4.2, Figure 5.2
II	Subform (parent–child data)	Y	Document_Request → Request_Items	4.2, 5.2	Figure 5.3
II	Field validations / layout rules	Y	Register_Number, Declaration, Purpose_Details layout rule	5.2	Figure 5.2
II	List report with filter, grouping, custom action	Y	All_Requests; Issued_Documents	5.3	Figure 5.5
II	Kanban / Calendar / Timeline / Map report	Y	Request_Pipeline; Due_Date_Calendar	5.3	Figure 5.6, Figure 5.7
II	Pivot table / chart report	Y	Requests_by_Document_Type; Monthly_Request_Trend	5.3	Figure 5.8, Figure 5.9
II	Page / dashboard built with UI Builder	Y	Admin_Dashboard; Student_Home; Verify_Document	5.4	Figure 5.10, Figure 5.11
II	Deluge – form events (validate / success / field actions)	Y	DS-01, DS-02, DS-04, DS-05	5.5	Code 5.1, Code 5.2
II	Deluge – functions, fetch, loops, collections	Y	DS-03, DS-10, DS-11 (Appendix C)	5.5	Code 5.3

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Email / in-app notifications	Y	DS-04, DS-05, DS-07	5.5, 5.6	Code 5.2
II	Form workflow	Y	Validation, Auto-fill, Fee, Acknowledgement, Issue	5.6	Table 5.3
II	Scheduled workflow	Y	SLA_Reminder; Expire_Documents	5.6	Table 5.3
II	Approval process	Y	Request Approval process	5.6	Figure 5.14
II	Blueprint	Y	Request Lifecycle Blueprint	5.6	Figure 5.13
II	Batch / report workflow	Y	Archive_Closed_Requests (monthly)	5.6	Table 5.3
II	Testing & debugging evidence	Y	Test cases TC-01 to TC-20; debug log	6	Table 6.1, Figure 6.1
III	Integration with a Zoho service	Y	Zoho Mail; Zoho Cliq	5.7	Figure 5.15
III	Connection / REST API call (invokeurl)	Y	QR-code API in DS-05	5.7	Code 5.3
III	Webhook or Creator REST API	Y	Creator REST API v2 (Postman)	5.7	Figure 5.15
III	Users, roles and permission profiles	Y	Four permission profiles; role hierarchy	5.8	Figure 5.16
III	Record-level access criteria / customer portal	Y	Student and HoD profiles; customer portal	5.8	Figure 5.16
III	Zia / AI feature	Y	Zia AI builder; AI Text Recognition field	5.9	Figure 5.17
III	Analytics (charts / Zoho Analytics)	Y	Charts, pivot report, Admin_Dashboard	5.9	Figure 5.10
IV	Performance optimisation applied & measured	Y	Section 7.2 optimisations	7.2	Table 7.2
IV	Deployment across environments	Y	Development → Stage → Production	7.1	Figure 7.1
IV	Security controls (encryption, OAuth, permissions)	Y	Role profiles, record-level criteria, encrypted field	7.3	Table 7.3

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
IV	Governance (audit trail, administration)	Y	Audit trail; Access_Log	7.4	Figure 7.2
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Not used – justified in Section 5.10	5.10	—
V	Low-code limitations & spectrum analysis	Y	Sections 8.4 and 8.5	8.4, 8.5	—
—	Mobile app / portal access	Y	Customer portal and mobile app	5.11	Figure 5.18

Total features implemented: 31 / 32

APPENDIX B – COURSE OUTCOME MAPPING

The table shows where each Course Outcome of CB23F35 is evidenced in this report and application. Attainment is self-assessed (1 – Low, 2 – Medium, 3 – High); the examiner may revise it.

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	3	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	3	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	3	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	3	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	2	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

All Deluge scripts of the application are given below, grouped by form or function, with a header comment giving the script ID (Table 5.2), the location or trigger and the purpose.

C.1 Document_Request – On Validate (DS-01)

```
// Script ID : DS-01
// Location  : Document_Request -> Workflow -> On Add -> On
Validate
// Purpose   : Block duplicate open requests and unrealistic
Required_By dates
if(input.Required_By != null && input.Required_By <
zoho.currentdate)
{
    alert "Required By date cannot be in the past.";
    cancel submit;
}
// longest processing time among the requested document types
maxDays = 0;
for each newRow in input.Request_Items
{
    if(newRow.Document_Type.Processing_Days > maxDays)
    {
        maxDays = newRow.Document_Type.Processing_Days;
    }
}
earliest = zoho.currentdate.addDay(maxDays);
if(input.Required_By < earliest)
{
    alert "The selected documents need " + maxDays + " day(s).
Choose " + earliest.toString("dd-MMM-yyyy") + " or later.";
    cancel submit;
}
// duplicate open request for the same document type
openReq = Document_Request[Student == input.Student && (Status ==
"Submitted" || Status == "Under Review" || Status == "Pending
Approval" || Status == "Approved" || Status == "In Preparation")];
for each req in openReq
{
    for each oldRow in req.Request_Items
    {
        for each newRow in input.Request_Items
        {
            if(oldRow.Document_Type == newRow.Document_Type)
            {
                alert "An open request already exists for " +
newRow.Document_Type.Document_Name + " (" + req.Request_ID + ").";
                cancel submit;
            }
        }
    }
}
```

C.2 Document_Request – On User Input of Student (DS-02)

```
// Script ID : DS-02
// Location  : Document_Request -> Workflow -> On User Input of
Student
// Purpose   : Auto-fill department, student e-mail and HoD e-mail
if(input.Student != null)
{
    stu = Student[ID == input.Student];
    dept = Department[ID == stu.Department];
    input.Department = stu.Department;
    input.Student_Email = stu.Email;
    input.HoD_Email = dept.HoD_Email;
}
else
{
    clear Department;
    clear Student_Email;
    clear HoD_Email;
}
```

C.3 Function calcTotalFee (DS-03)

```
// Script ID : DS-03
// Location  : Functions -> calcTotalFee (called from DS-04)
// Purpose   : Return the total fee of a saved request from its
subform rows
decimal calcTotalFee(int requestID)
{
    total = 0.0;
    req = Document_Request[ID == requestID];
    for each row in req.Request_Items
    {
        total = total + (row.Document_Type.Fee * row.Copies);
    }
    return total;
}
```

C.4 Document_Request – On Success (DS-04)

```
// Script ID : DS-04
// Location  : Document_Request -> Workflow -> On Add -> On
Success
// Purpose   : Send an acknowledgement e-mail to the student and
alert the office on Zoho Cliq
stu = Student[ID == input.Student];
fee = thisapp.calcTotalFee(input.ID);
docList = List();
for each row in input.Request_Items
{
    docList.add(row.Document_Type.Document_Name + " x " +
row.Copies);
}
sendmail
[
    from :zoho.adminuserid
    to :input.Student_Email
```

```
    subject :"DocuVault: request " + input.Request_ID + "
received"
    message :"Dear " + stu.Student_Name + ",<br>Your request for "
+ docList.toString(", ") + " has been received.<br>Total fee: Rs.
" + fee + "<br>Required by: " + input.Required_By.toString("dd-
MMM-yyyy") + "<br>You can track the status on the DocuVault
student portal."
]
zoho.cliq.postToChannel("docuvault-office", "New request " +
input.Request_ID + " from " + stu.Student_Name + " (Rs. " + fee +
")", "cliq_connection");
```

C.5 Issued_Document – On Success (DS-05)

```
// Script ID : DS-05
// Location  : Issued_Document -> Workflow -> On Add -> On Success
// Purpose   : Generate verification code, validity date and QR
code; notify the student
rec = Issued_Document[ID == input.ID];
req = Document_Request[ID == input.Request];
dtype = Document_Type[ID == input.Document_Type];
stu = Student[ID == req.Student];
// unique, non-guessable verification code
hash = zoho.encryption.sha256(input.ID.toString() +
zoho.currenttime.toString());
code = "DV-" + zoho.currentdate.toString("yyyy") + "-" +
hash.substring(0,8).toUpperCase();
validTill = zoho.currentdate.addDay(dtype.Validity_Days);
// QR code of the public verification link
verifyUrl = " https://creatorapp.zoho.in/[your-published-page-
link]?code=" + code;
qrResp = invokeurl
[
    url :"https://api.qrserver.com/v1/create-qr-
code/?size=200x200&data=" + encodeUrl(verifyUrl)
    type :GET
];
rec.Verification_Code = code;
rec.Valid_Until = validTill;
rec.Verification_QR = qrResp.toFile("QR_" + code + ".png");
rec.Issued_By = zoho.loginuser;
sendmail
[
    from :zoho.adminuserid
    to :stu.Email
    subject :"Your " + dtype.Document_Name + " is ready (" + code
+ ")"
    message :"Dear " + stu.Student_Name + ",<br>Your " +
dtype.Document_Name + " is ready. Download it from the student
portal before " + validTill.toString("dd-MMM-yyyy") +
".<br>Verification code: " + code
]
```

C.6 Blueprint transition "Issue Document" – Before (DS-06)

```
// Script ID : DS-06
```

```
// Location  : Blueprint "Request Lifecycle" -> Transition "Issue
Document" -> Before
// Purpose   : Allow the transition only when every requested item
has an active issued document
thisReq = Document_Request[ID == input.ID];
requestedCount = thisReq.Request_Items.count();
issuedCount = Issued_Document[Request == input.ID && Doc_Status ==
"Active"].count();
if(issuedCount < requestedCount)
{
    alert "Upload an issued document for every requested item
before marking the request as Issued.";
    cancel submit;
}
```

C.7 Schedule – Daily SLA reminder (9:00 AM) (DS-07)

```
// Script ID : DS-07
// Location  : Workflow -> Schedules -> SLA_Reminder (daily, 9:00
AM)
// Purpose   : E-mail a digest of requests that exceeded their
processing time
overdue = List();
openReq = Document_Request[Status != "Issued" && Status !=
"Closed" && Status != "Rejected" && Archived == false];
for each req in openReq
{
    age = daysBetween(req.Request_Date, zoho.currenttime);
    maxDays = 0;
    for each row in req.Request_Items
    {
        if(row.Document_Type.Processing_Days > maxDays)
        {
            maxDays = row.Document_Type.Processing_Days;
        }
    }
    if(age > maxDays)
    {
        overdue.add(req.Request_ID + " - " +
req.Student.Student_Name + " (" + age + " days)");
    }
}
if(overdue.size() > 0)
{
    sendmail
    [
        from :zoho.adminuserid
        to :zoho.adminuserid
        subject :"DocuVault: " + overdue.size() + " request(s)
past their processing time"
        message :overdue.toString("<br>")
    ]
    zoho.cliq.postToChannel("docuvault-office", "SLA alert: " +
overdue.size() + " request(s) overdue", "cliq_connection");
}
```

C.8 Schedule – Daily document expiry (12:30 AM) (DS-08)

```
// Script ID : DS-08
// Location  : Workflow -> Schedules -> Expire_Documents (daily,
12:30 AM)
// Purpose   : Expire active documents whose validity date has
passed and log the event
expiring = Issued_Document[Doc_Status == "Active" && Valid_Until <
zoho.currentdate];
for each doc in expiring
{
    doc.Doc_Status = "Expired";
    insert into Access_Log
    [
        Added_User = zoho.loginuser
        Issued_Document = doc.ID
        Action = "Expired"
        Accessed_By = "System (Schedule)"
        Accessed_On = zoho.currenttime
        Result = "Auto-expired"
    ]
}
```

C.9 Batch workflow – Archive closed requests (DS-09)

```
// Script ID : DS-09
// Location  : Workflow -> Batch -> Archive_Closed_Requests
(monthly, 1st, 2:00 AM)
// Purpose   : Archive requests that have been closed for more
than 12 months
cutoff = zoho.currentdate.subMonth(12);
oldClosed = Document_Request[Status == "Closed" && Request_Date <
cutoff && Archived == false];
for each req in oldClosed
{
    req.Archived = true;
}
info "Archived " + oldClosed.count() + " request(s)";
```

C.10 Function verifyDocument (DS-10)

```
// Script ID : DS-10
// Location  : Functions -> verifyDocument (called from the
Verify_Document page)
// Purpose   : Return the validity details of a document for a
verification code
map verifyDocument(string code)
{
    result = Map();
    doc = Issued_Document[Verification_Code ==
code.trim().toUpperCase()];
    if(doc.count() == 0)
    {
        result.put("valid", false);
        result.put("message", "No document found for this code.");
        return result;
    }
```

```
    stu = Student[ID == doc.Student];
    isValid = (doc.Doc_Status == "Active");
    result.put("valid", isValid);
    result.put("document", doc.Document_Type.Document_Name);
    result.put("student", stu.Student_Name);
    result.put("issued_on", doc.Issue_Date.toString("dd-MMM-
yyyy"));
    result.put("message", if(isValid, "This document is genuine
and active.", "This document is " + doc.Doc_Status + "."));
    insert into Access_Log
    [
        Added_User = "Public Verifier"
        Issued_Document = doc.ID
        Action = "Verified"
        Accessed_By = "Public Verifier"
        Accessed_On = zoho.currenttime
        Result = if(isValid, "Valid", doc.Doc_Status)
    ]
    return result;
}
```

C.11 Function logAccess (DS-11)

```
// Script ID : DS-11
// Location  : Functions -> logAccess (called by the custom action
"Download & Log")
// Purpose   : Write an Access_Log entry and increment the
download counter
void logAccess(int docId, string action)
{
    doc = Issued_Document[ID == docId];
    if(doc.Doc_Status != "Active")
    {
        return;
    }
    insert into Access_Log
    [
        Added_User = zoho.loginuser
        Issued_Document = docId
        Action = action
        Accessed_By = zoho.loginuserid
        Accessed_On = zoho.currenttime
        Result = "Success"
    ]
    doc.Download_Count = ifnull(doc.Download_Count, 0) + 1;
}
```

C.12 Custom action “Mark Paid” (DS-12)

```
// Script ID : DS-12
// Location  : Report All_Requests -> Custom action "Mark Paid"
// Purpose   : Record that the fee of the selected request has
been received
rec = Document_Request[ID == input.ID];
if(rec.Payment_Status == "Paid")
{
    return;
```

```
}
rec.Payment_Status = "Paid";
rec.Remarks = ifnull(rec.Remarks, "") + " Fee received on " +
zoho.currentdate.toString("dd-MMM-yyyy") + ".";
```

APPENDIX D – USER MANUAL

This manual gives short, step-by-step instructions for each role so that a new user can operate DocuVault without help.

D.1 Accessing the Application

Open the application link given in "Project at a Glance" in a browser, or install the Zoho Creator mobile app and sign in. Staff and HoDs sign in with their invited Zoho account; students sign in to the customer portal with the e-mail address registered in the Student form (use *Forgot password* on first login). After login each user sees only the menu sections permitted by their profile.

D.2 Instructions for the Student

- Sign in and open *Student_Home*; select *Request a Document*.
- Select your name; the department and e-mail fill automatically.
- Add one row for each document (document type and copies); the fee is shown.
- Choose the purpose, the required-by date and the delivery mode, attach a photo of your ID card, tick the declaration and press *Submit Request*.
- Track the request in *My_Requests*; you receive an e-mail at each important step.
- When the "document ready" e-mail arrives, open *My_Documents* and use *Download & Log* before the validity date.

D.3 Instructions for the Office Staff

- Open *Request_Pipeline* or *All_Requests* to see new requests; open a request and use the Blueprint button *Start Review.*
- Compare the ID-card text with the student record, use *Mark Paid* when the fee is received, and select *Submit for Approval*.
- After approval, select *Start Preparation*, prepare the signed PDF and add an *Issued_Document* record with the file.
- When all items are uploaded, select *Issue Document*, and later *Close* after the student has collected or downloaded the documents.
- Check *Due_Date_Calendar* and the daily reminder e-mail for overdue requests.

D.4 Instructions for the HoD

- Open the approval e-mail or *Pending_Approvals* to see requests of your department.
- Review the student, documents and purpose, then press *Approve* or *Reject*; give a reason when rejecting.

D.5 Instructions for the Administrator

- Maintain the masters (*Department*, *Student*, *Document_Type*) – for example, change a fee, processing days or validity.
- Approve high-value documents at the second approval level.
- Use *Admin_Dashboard* and the Analytics reports to monitor volume and turnaround; review *Access_Log_Report* for audit.
- Manage users and profiles, and export the data periodically.

D.6 Instructions for a Verifier

- Open the Verify_Document page or scan the QR code printed on the document.
- Enter the verification code and press *Verify*; the page shows whether the document is genuine and active, with the student name, document type and issue date.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	Nethra G	Register No.	240701358
Project Title	DocuVault – Digital Document Request & Retrieval System	Date	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	Nethra G	Register No.	240701358
Department / Section	Computer Science and Engineering	Date of Viva	11.11.2026
Application URL Verified	Yes	Demo Duration	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Name: _____ Name: _____